



Universidad de Málaga

Escuela de Ingenierías Industriales

Ingeniería de Sistemas y Automática

Trabajo Fin de Grado

Prototipo mecatrónico de salto controlado en altura y dirección

Grado en Ingeniería Electrónica, Robótica y Mecatrónica

Autor: Jose Segura Montes

Tutor: Jesus Manuel Gómez de Gabriel

19 de junio de 2026

Declaración de Originalidad del Trabajo

Fin de Grado

D./Dña. Jose Segura Montes

DNI/Pasaporte: 77644619T. Correo electrónico: jose.segura.montes@gmail.com

Titulación: Grado en Ingeniería Electrónica, Robótica y Mecatrónica

Título del Proyecto/Trabajo: Prototipo mecatrónico de salto controlado en altura y dirección

DECLARA BAJO SU RESPONSABILIDAD

Ser autor/a del texto entregado y que no ha sido presentado con anterioridad, ni total ni parcialmente, para superar materias previamente cursadas en esta u otras titulaciones de la Universidad de Málaga o cualquier otra institución de educación superior u otro tipo de fin.

Asimismo, declara no haber trasgredido ninguna norma universitaria con respecto al plagio ni a las leyes establecidas que protegen la propiedad intelectual, así como que las fuentes utilizadas han sido citadas adecuadamente.

En Málaga, a 19 de junio de 2026

Fdo.: Jose Segura Montes

Resumen

Prototipo mecatrónico de salto controlado en altura y dirección

Autor: Jose Segura Montes

Tutor: Jesus Manuel Gómez de Gabriel

Departamento: Ingeniería de Sistemas y Automática

Titulación: Grado en Ingeniería Electrónica, Robótica y Mecatrónica

Palabras clave Robot cuadrúpedo, Salto controlado, Robótica móvil, Diseño mecatrónico, Actuadores elásticos, Dinámica del salto

Este Trabajo de Fin de Grado aborda el diseño, modelización, construcción, programación y experimentación de un prototipo de mecanismo de salto para robots autónomos cuadrúpedos. El punto de partida es la limitación clásica de los robots de locomoción terrestre: la incapacidad de superar obstáculos con continuidad y eficiencia. La solución propuesta integra un mecanismo de salto controlado en altura sobre un robot cuadrúpedo con patas romboidales, gestionado desde un ESP32 mediante micro-ROS y una plataforma de control en ROS 2 C++.

Se han diseñado y comparado tres mecanismos de salto originales. El primero, basado en tornillo sin fin, se descartó por su baja eficiencia mecánica cuando es fabricado en impresión 3D. El segundo, de trinquete y lengüeta (*ratchet and pawl*) con reductora, demostró ser funcional pero presentó problemas de fatiga en piezas impresas y resolución discreta de compresión. El tercer mecanismo, de doble muelle en paralelo con activación directa mediante carrete y segundo servo de disparo, resultó ser el más robusto, sencillo y reproducible, y es sobre el que se han realizado los experimentos.

El modelado teórico se apoya en dos bloques: tiro parabólico en dos dimensiones para obtener la velocidad y el ángulo óptimos de despegue, y conservación de energía para determinar la compresión necesaria del muelle en función del obstáculo. Ambos se resuelven numéricamente en C++ dentro del nodo ROS 2. La cinemática inversa de las cuatro patas romboidales permite al robot inclinarse al ángulo calculado antes de efectuar el salto.

Los resultados preliminares validan el modelo teórico y demuestran la capacidad del mecanismo de superar obstáculos de altura variable. El trabajo identifica además las principales vías de mejora: integración de sensores de posición, y control aéreo y autonomía del robot.

Agradecimientos

Se lo dedico a toda mi familia, que ha sabido apoyarme y guiarme a lo largo de mi trayectoria y, lo más importante, me ha dejado equivocarme.

Acrónimos y Notación Matemática

CAD	Diseño Asistido por Ordenador (<i>Computer-Aided Design</i>)
COM	Centro de Masas (<i>Center of Mass</i>)
DC	Corriente Continua (<i>Direct Current</i>)
DDS	Servicio de Distribución de Datos (<i>Data Distribution Service</i>)
EESR	Almacenamiento y Liberación de Energía Elástica (<i>Elastic Energy Storage and Release</i>)
IK	Cinemática Inversa (<i>Inverse Kinematics</i>)
LaMSA	Actuación por Muelle Mediada por Cerrojo (<i>Latch-Mediated Spring Actuation</i>)
MIT	Instituto Tecnológico de Massachusetts (<i>Massachusetts Institute of Technology</i>)
PETG	Tereftalato de Polietileno Glicol (<i>Polyethylene Terephthalate Glycol</i>)
PLA	Ácido Poliláctico (<i>Polylactic Acid</i>)
PWM	Modulación por Ancho de Pulsos (<i>Pulse Width Modulation</i>)
ROS 2	Sistema Operativo Robótico versión 2 (<i>Robot Operating System 2</i>)
SDK	Kit de Desarrollo de Software (<i>Software Development Kit</i>)
SEA	Actuador Elástico en Serie (<i>Series Elastic Actuator</i>)
TFG	Trabajo de Fin de Grado
ToF	Tiempo de Vuelo (<i>Time of Flight</i>)
TTL	Lógica Transistor-Transistor (<i>Transistor-Transistor Logic</i>)
UART	Transmisor-Receptor Asíncrono Universal (<i>Universal Asynchronous Receiver-Transmitter</i>)
D	Distancia horizontal de despegue al obstáculo
g	Aceleración de la gravedad ($9,81 \text{ m/s}^2$)
h	Altura objetivo del salto
k	Constante elástica del muelle (N/m)
M	Masa total de la plataforma robótica
v_0	Velocidad inicial de despegue del centro de masas
Y_{prep}	Altura dinámica de preparación del chasis previa al salto
Z_0	Altura inicial del centro de masas en el instante de despegue
Δx	Elongación o compresión lineal del muelle
η	Factor de eficiencia mecánica y de transferencia de energía
θ	Ángulo óptimo de lanzamiento respecto a la horizontal
μ	Coefficiente de rozamiento estático entre el pie y el suelo
τ_{nom}	Par motor nominal de los servomotores

Índice

	Página
Índice de Figuras	xv
Índice de Tablas	xvii
1 Introducción	1
1.1. Motivación	1
1.2. Introducción a los robots saltadores	2
1.3. Objetivos	2
1.3.1. Objetivo general	2
1.3.2. Objetivos específicos	3
1.4. Alcance y limitaciones del trabajo	3
1.5. Robots de salto: revisión del estado del arte	3
1.6. Almacenamiento y liberación de energía elástica	4
1.7. Mecanismos de retención y disparo	5
1.8. Robots cuadrúpedos con patas romboidales	6
1.9. ROS 2 y micro-ROS en sistemas embebidos	6
2 Diseño del sistema	9
2.1. Modelado teórico del salto	9
2.1.1. Hipótesis del modelo	9
2.1.2. Tiro parabólico	10
2.1.3. Restricción de rozamiento	10
2.1.4. Conservación de la energía	11
2.1.5. Adaptación dinámica de la altura de preparación	11
2.1.6. Cálculo del tiempo de compresión	11
2.1.7. Implementación numérica	12
2.2. Diseño y comparativa de los tres mecanismos de salto	15
2.2.1. Criterios de diseño comunes	15
2.2.2. Mecanismo 1: tornillo sin fin	15

2.2.3.	Mecanismo 2: trinquete y lengüeta con reductora	16
2.2.4.	Mecanismo 3: doble muelle con disparo por desenganche directo (seleccionado)	19
2.3.	Diseño del robot cuadrúpedo	21
2.3.1.	Topología del robot	21
2.3.2.	Dimensionado de las patas	22
2.3.3.	Selección final del elemento elástico	23
2.3.4.	Cinemática inversa de las patas	24
2.3.5.	Inclinación del chasis para el salto	25
2.4.	Arquitectura de software	26
2.4.1.	Visión general	26
2.4.2.	Tópicos ROS 2	26
2.4.3.	Máquina de estados del nodo principal	27
2.4.4.	Nodo esclavo ESP32	27
2.4.5.	Script de teleoperación	28
2.5.	Proceso de montaje y calibración	28
2.5.1.	Asignación de IDs a los servos	28
2.5.2.	Calibración de rangos de los servos de las patas	28
2.5.3.	Problemas encontrados durante el montaje	28
3	Implementación	31
3.1.	Enfoque de diseño	31
3.2.	Lista de materiales (BOM) y presupuesto	32
3.3.	Herramientas de software	33
3.4.	Montaje físico final	33
3.5.	Criterios de validación	34
4	Experimentación	37
4.1.	Metodología de medición	37
4.2.	Calibración del factor de eficiencia (η)	38
4.3.	Validación del salto vertical a diferentes alturas	40
4.3.1.	Parámetros del sistema	40
4.3.2.	Resultados experimentales	40
4.3.3.	Análisis de precisión y fuentes de error	42
5	Discusión y Conclusiones	43
5.1.	Conclusiones	43
5.2.	Limitaciones del trabajo	44
5.3.	Trabajo futuro	45
A	Código completo del nodo principal (robot.hpp)	47

B Código principal del nodo Robot (robot.cpp)	55
C Código del nodo esclavo ESP32	71
D Comandos de referencia rápida	73
Referencias	75

Índice de Figuras

Figura	Página
2.1. Esquema de tiro parabólico	10
2.2. Mecanismo número 1. Implementación física. Planta	15
2.3. Mecanismo número 1. Implementación física. Perfil	16
2.4. Mecanismo número 1. Implementación física. Carga del muelle	16
2.5. Mecanismo Ratchet And Pawl. SolidWorks	17
2.6. Mecanismo Ratchet And Pawl. SolidWorks. Vista desde Planta	17
2.7. Mecanismo Ratchet And Pawl. SolidWorks. Vista desde Perfil	18
2.8. Ensamblaje del mecanismo 3. Solidworks.	19
2.9. Ensamblaje del mecanismo 3. Solidworks. Parte trasera.	20
2.10. Ensamblaje completo. Solidworks.	22
3.1. Montaje final. Vista de frente	34
3.2. Montaje final. Vista de perfil	34
4.1. Experimento número 1. Fotograma de altura máxima. $\eta = 1$	38
4.2. Experimento número 1. Fotograma de altura máxima. $\eta = 0,9$	39
4.3. Experimento número 2. Fotograma de altura máxima. $\eta = 0,7$	39
4.4. Análisis de la altura del COM con $h = 5cm$. Kinovea	41
4.5. Análisis de la altura del COM con $h = 4cm$. Kinovea	41

Índice de Tablas

Tabla	Página
2.1. Parámetros geométricos de las patas	22
2.2. Tópicos ROS 2 del sistema	26
3.1. Lista de materiales (BOM) y presupuesto de ejecución material del prototipo	32
4.1. Resultados de la calibración de la eficiencia (η) para un salto objetivo de $h = 4 - 5$ cm (media de 3 ensayos)	38
4.2. Parámetros físicos constantes utilizados en el algoritmo del robot	40
4.3. Análisis cinemático y estadístico de los saltos verticales (media de 3 ensayos)	42

Introducción

1.1. Motivación

La robótica móvil terrestre lleva décadas dominada por robots con ruedas o con patas, cada uno con sus ventajas e inconvenientes. Los robots de ruedas son eficientes y rápidos en superficies lisas, pero quedan completamente bloqueados ante un escalón o un obstáculo de cierta altura. Los robots con patas ofrecen mucha más versatilidad de movimiento, pero la mayoría siguen sin poder superar barreras verticales significativas sin un mecanismo adicional.

Este proyecto nace precisamente de esa limitación. La idea es dotar a un robot cuadrúpedo de la **capacidad de saltar un obstáculo de altura variable**, controlada por software, sin comprometer el resto de sus capacidades locomotoras. No se trata de un sistema de salto fijo como los que se encuentran en mecanismos de leva o engranaje cortado, sino de uno que calcula en tiempo real la energía necesaria a emplear en función de la altura del obstáculo que se quiere superar.

Todo esto permite crear soluciones adaptadas y personalizadas para superar obstáculos dinámicamente, usando componentes y dispositivos de baja potencia, logrando un resultado equivalente a el uso de motores de locomoción de mucha potencia y consumo.

Desde un punto de vista personal, la elección de este tema responde al interés en combinar todas las áreas presentes en la robótica y en la mecatrónica, como pueden ser el diseño mecánico, modelización física, impresión 3D, electrónica embebida, programación y experimentación en un solo prototipo funcional que tenga un problema real que resolver. El resultado es un sistema que integra física clásica, cinemática inversa y comunicación distribuida en tiempo real.

1.2. Introducción a los robots saltadores

La locomoción en robótica móvil se ha resuelto tradicionalmente mediante ruedas o cadenas, soluciones muy eficientes en superficies planas pero que fracasan ante obstáculos verticales que superan el radio de la rueda. Para entornos irregulares o inexplorados, los robots con patas ofrecen una adaptabilidad mucho mayor. Sin embargo, cuando el tamaño del robot es reducido, incluso caminar resulta insuficiente para superar escalones, rocas o zanjás. Es en este contexto donde el salto se presenta como una solución de movilidad, permitiendo a plataformas de pequeña escala superar obstáculos que multiplican varias veces su propio tamaño [1].

A pesar de sus evidentes ventajas tácticas, replicar esta habilidad biológica en una máquina entraña una enorme complejidad mecatrónica. Diseñar un robot saltador no consiste simplemente en añadir motores más potentes a un chasis convencional, sino que requiere resolver tres grandes retos de ingeniería [2]:

1. **El límite de potencia térmica y mecánica:** Un salto efectivo exige acelerar la masa del robot en una fracción de segundo. Los actuadores eléctricos directos (como los servomotores comunes) que son lo suficientemente ligeros para ir a bordo de un microrobot no pueden generar este pico de potencia instantáneo sin quemarse. Esto obliga a diseñar arquitecturas mecánicas intermedias, como resortes o sistemas neumáticos, para acumular la energía lentamente y liberarla de golpe.
2. **La gestión del impacto:** Lo que sube, tiene que bajar. Durante el aterrizaje, la estructura del robot y, sobre todo, los engranajes internos de los motores sufren picos de fuerza que pueden destruir los componentes fácilmente. El diseño físico debe contemplar cierta flexibilidad o mecanismos de absorción de impactos para sobrevivir a ciclos de saltos repetidos.
3. **El control y la estabilidad:** Durante la fase de vuelo, el robot se convierte en un sistema subactuado (no tiene puntos de apoyo para corregir su trayectoria). Cualquier pequeña asimetría en el momento del despegue, como un resbalón de una pata o un mal reparto de masas, se traduce en un giro incontrolado en el aire.

Por tanto, el desarrollo de este tipo de plataformas exige un enfoque holístico donde la selección de materiales, la cinemática de las patas y el control del software deben diseñarse en conjunto para encontrar un punto de equilibrio funcional.

1.3. Objetivos

1.3.1. Objetivo general

Diseñar, construir y validar experimentalmente un prototipo de mecanismo de salto integrado en un robot cuadrúpedo, con control de la altura del salto por software.

1.3.2. Objetivos específicos

- Diseñar al menos dos mecanismos de salto alternativos y comparar su viabilidad teórica y práctica.
- Desarrollar un modelo teórico del salto basado en tiro parabólico y conservación de energía que permita calcular los parámetros necesarios (velocidad, ángulo y compresión del muelle) en función de la geometría del obstáculo.
- Diseñar e implementar la cinemática inversa de las patas romboidales del robot para permitir la inclinación del chasis al ángulo óptimo de lanzamiento.
- Programar la lógica de control completa en ROS 2 sobre una arquitectura maestro (PC) – esclavo (ESP32 con micro-ROS).
- Realizar experimentos de salto sobre obstáculos de altura variable y contrastar el modelo teórico con los resultados obtenidos.

1.4. Alcance y limitaciones del trabajo

El prototipo desarrollado es un primer robot de demostración. Se han asumido varias simplificaciones que acoten el problema a un tamaño manejable para un TFG:

- El salto se restringe a dos dimensiones: solo se efectúa en el plano XZ , es decir, únicamente en la dirección vertical y hacia adelante.
- El robot se trata como una partícula centrada en su centro de masa (COM) durante la fase de vuelo, lo que es razonable dado que la fuerza de salto pasa prácticamente por el COM y el momento de inercia del chasis dificulta la rotación.
- El sistema de almacenamiento de energía y transición de fase (lógica del robot) se realizan en bucle abierto, para ahorrar tanto en costes económicos como en peso del robot.

1.5. Robots de salto: revisión del estado del arte

Los mecanismos de salto en robótica han sido objeto de estudio desde la década de los 80. El referente histórico más claro es el Raibert Hopper [3], un robot monopata que sentó las bases del equilibrio dinámico y el salto continuo en bucle cerrado. A partir de este hito, la investigación se **diversificó** buscando superar las limitaciones mecánicas de los motores rígidos tradicionales, a menudo inspirándose en la naturaleza.

Por un lado, surgieron investigaciones y diseños enfocados puramente en optimizar el salto mediante distintas estrategias de **acumulación de energía**. Destacan los actuadores elásticos en

serie (SEA), que imitan el comportamiento de los tendones para modular la potencia mecánica, permitiendo saltos continuos y ágiles como es el caso del robot Salto-1P [4]. Otras líneas han apostado por la multiplicación de trabajo mediante mecanismos de trinquete para acumular energía durante más tiempo y liberarla de golpe, batiendo récords de altura y superando los límites biológicos [5]. En paralelo, se han documentado enfoques alternativos como el uso de pandeo estructural (*snap-through*) en robótica blanda [6] o los sistemas de combustión y expansión de gases pensados para elevar plataformas pesadas de un solo impulso [7].

A pesar de estos avances en salto puro, la integración de esta habilidad en robots caminantes plantea retos muy distintos. En los últimos años, la industria y la academia han convergido hacia soluciones cuadrúpedas de **alta agilidad**, como el Cheetah del MIT [8] o el ANYmal de ANYbotics [9]. Estos sistemas logran saltar obstáculos considerables y aterrizar de forma estable, pero dependen de actuadores *quasi-direct drive* de diseño propio, hardware de alto coste y alto consumo y requerimientos computacionales muy restrictivos.

En este contexto, lo que distingue al enfoque de este trabajo respecto a la literatura dominante es la búsqueda de un **equilibrio** entre la funcionalidad dinámica y la accesibilidad, combinando tres características simultáneas:

1. El salto se integra en un robot cuadrúpedo que también **camina**, sin perder esa capacidad de locomoción básica.
2. La altura del salto es **parametrizable** por software en tiempo real, adaptándose a las necesidades del entorno.
3. Todo el sistema se construye con componentes de **bajo coste** (impresión 3D, servomotores económicos, ESP32) y herramientas de código abierto (ROS 2, micro-ROS), lo que democratiza el acceso a este tipo de plataformas.

1.6. Almacenamiento y liberación de energía elástica

La estrategia más extendida para conseguir un salto de alta aceleración con actuadores de baja potencia es **almacenar energía en un elemento elástico** (ya sea un muelle metálico o un material compuesto) y liberarla instantáneamente. Este principio, muy presente en la naturaleza en animales como saltamontes, ranas y pulgas, se denomina *catapulting* o *elastic energy storage and release* (EESR) [10].

El uso de este principio resuelve el principal problema físico del salto: la **limitación de potencia de los motores**. Un motor eléctrico convencional, especialmente uno de bajo coste, no puede entregar la fuerza y velocidad instantáneas necesarias para elevar la masa del robot en el aire. Mediante el enfoque EESR, el motor trabaja durante un periodo de tiempo más largo acumulando energía de forma gradual. Cuando el mecanismo se libera, esa energía se transmite al suelo en apenas unos

milisegundos. A este efecto multiplicador se le conoce como amplificación de potencia (*power amplification*) [11].

En este trabajo se emplea un muelle helicoidal de tracción como elemento de almacenamiento. La elección frente a los tradicionales muelles de compresión se justifica por dos motivos principales. **En primer lugar**, por el diseño propio del mecanismo de carrete: el hilo enrollado tira del extremo inferior del muelle, estirándolo linealmente, y al soltarse, transfiere el impulso directamente a la pata de salto. **En segundo lugar**, los muelles de tracción evitan el problema del pandeo (*buckling*), una tendencia natural de los muelles de compresión a doblarse hacia los lados cuando se someten a mucha carga, lo que obligaría a añadir guías o ejes extra que aumentarían el peso del conjunto.

Además, el uso de un muelle lineal facilita el control del salto. La energía potencial elástica almacenada (E) sigue la ley de Hooke básica, descrita como $E = \frac{1}{2}k\Delta x^2$, donde k es la constante elástica del muelle y Δx es el desplazamiento del hilo. Esta relación matemática permite que el sistema calcule exactamente cuánto debe enrollar el carrete para alcanzar una **altura de salto específica**.

1.7. Mecanismos de retención y disparo

Para controlar el momento exacto de la liberación de la energía elástica se necesita un mecanismo de retención o *latch*. Como señala el marco de trabajo de la actuación por muelle mediada por cerrojo (LaMSA, por sus siglas en inglés) [12], el diseño de este componente es crítico: no solo debe aguantar la carga del muelle sin ceder, sino que debe poder liberarse de forma casi instantánea para no disipar la energía acumulada en forma de fricción. En la literatura robótica se encuentran tres grandes familias para resolver este problema:

Mecanismos de trinquete y lengüeta (*ratchet and pawl*): Permiten cargar el muelle de forma incremental, diente a diente. La lengüeta bloquea mecánicamente el retroceso del sistema, por lo que el motor principal no necesita ejercer fuerza continua para mantener la compresión. Para disparar el salto, un actuador secundario retira la lengüeta. Su principal limitación es que ofrecen una resolución discreta (limitada por el tamaño de los dientes) y el mecanismo puede sufrir atascos o requerir mucha fuerza de liberación si la tensión acumulada es muy alta.

Levas y engranajes incompletos (*slip gears*): Son muy comunes en robots de salto de pequeña escala inspirados en ranas o insectos [13]. Consisten en un engranaje al que le falta una sección de dientes. El motor gira en un solo sentido, tensando el muelle mientras los dientes están engranados. Al llegar a la zona lisa, la transmisión pierde contacto y el muelle se dispara automáticamente. Aunque es un diseño muy ligero que usa un solo motor para cargar y soltar, la geometría física de la pieza define el ciclo por completo. Además, su limitación es clara, **solo permiten un almacenamiento de energía estático**.

Mecanismos de desenganche directo o embragues: Utilizan un sistema de acoplamiento que puede ser desconectado a voluntad, habitualmente mediante un servomotor secundario que retira un pasador o separa un embrague de fricción [14]. Aunque añaden masa extra al sistema y exigen mayor coordinación de control, permiten desenganchar el tren de transmisión en cualquier punto del recorrido del muelle. **Esta es la solución adoptada** en el tercer mecanismo de este trabajo, ya que es el único enfoque viable para cumplir con el objetivo de parametrizar la altura del salto por software en tiempo real.

1.8. Robots cuadrúpedos con patas romboidales

Las patas en **forma de rombo**, también conocidas como **mecanismos de cinco barras, pantográficas** o de **eslabonamiento paralelo**, son una elección habitual en el diseño de robots cuadrúpedos de pequeña escala debido a su excelente equilibrio entre simplicidad mecánica y rendimiento dinámico. La estructura básica consta de dos eslabones activos (conectados directamente a los ejes de los motores en la base) y dos eslabones pasivos que se unen en el extremo libre, formando una geometría cerrada que permite desplazar la punta de la pata en un plano vertical utilizando únicamente dos servomotores.

Una de las mayores ventajas de esta configuración geométrica frente a las patas de diseño serial (donde un motor se monta físicamente sobre el eslabón anterior, imitando una rodilla tradicional) es la **distribución de masas**. Al concentrar el peso de los actuadores en el cuerpo central del robot —o en la “cadera”—, la inercia de las partes móviles de la pata se reduce drásticamente [15]. Esta ligereza en el extremo distal es fundamental para realizar **movimientos muy rápidos con poco gasto energético y minimizar el impacto** durante las fases de aterrizaje en los saltos.

Además, desde la perspectiva del software y el control, la cinemática inversa de este tipo de pata tiene una **solución analítica geométrica directa**. Esto significa que el sistema no necesita realizar cálculos iterativos complejos para averiguar qué ángulo debe darle a los motores para poner el pie en una coordenada concreta. Esta ligereza de cálculo es clave para la viabilidad de plataformas accesibles como la de este trabajo, ya que permite ejecutar los algoritmos de locomoción a alta frecuencia utilizando microcontroladores económicos y de recursos limitados [16].

1.9. ROS 2 y micro-ROS en sistemas embebidos

ROS 2 (*Robot Operating System 2*) se ha consolidado como el estándar *de facto* para el desarrollo de sistemas robóticos distribuidos. Frente a su predecesor, ROS 1, esta nueva versión ofrece soporte nativo para comunicaciones en tiempo real apoyándose en el protocolo DDS (*Data Distribution Service*). Además, introduce mejoras sustanciales como la gestión del ciclo de vida de los nodos y una mayor seguridad en las comunicaciones de red [17].

Históricamente, la integración entre el cerebro principal de un robot (típicamente un ordenador o Raspberry Pi) y la electrónica de bajo nivel obligaba a escribir protocolos serie a medida para

intercambiar datos, lo que complicaba el código y su mantenimiento. Para solucionar esta brecha se utiliza micro-ROS [18]. Esta herramienta **extiende todo el ecosistema de ROS 2 a microcontroladores** de recursos limitados mediante una arquitectura cliente-agente. El agente actúa como puente, traduciendo el protocolo DDS estándar hacia conexiones más ligeras como UART o WiFi, lo que permite que el microcontrolador deje de ser un dispositivo aislado y pase a comportarse como un nodo nativo más dentro de la red del robot.

En este trabajo, el microcontrolador ESP32 aprovecha esta tecnología para integrarse en el sistema de forma directa. Actúa ejecutando micro-ROS para recibir las órdenes de posición de los servos publicadas por el nodo principal (PC) y, utilizando el mismo canal estandarizado, publica confirmaciones de estado y datos de vuelta, logrando un control del sistema **unificado, robusto y fácil de escalar**.

Diseño del sistema

2.1. Modelado teórico del salto

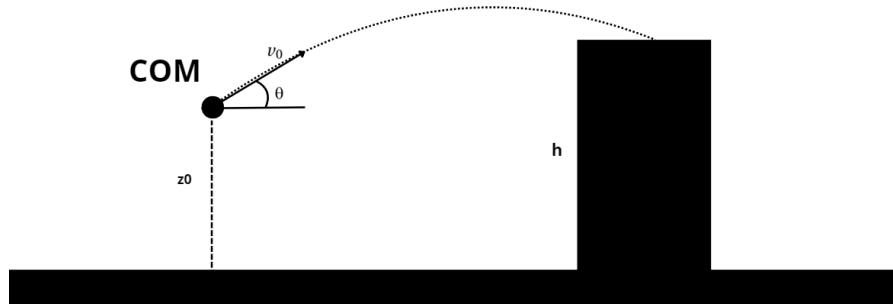
El modelo teórico se divide en **dos partes**: la trayectoria de vuelo (tiro parabólico) y la energía necesaria para conseguirla (conservación de la energía elástica) [19]. Ambas se resuelven numéricamente en el código de control para poder adaptar el salto en tiempo real.

2.1.1. Hipótesis del modelo

- El robot se modela como una **partícula de masa M concentrada en su centro de masas (COM)**. Esto es válido porque: (a) la fuerza de salto se aplica lo más cerca posible del COM, minimizando el par resultante; (b) el momento de inercia del chasis es suficientemente grande como para que la rotación angular durante el vuelo sea despreciable.
- El salto **se restringe al plano vertical XZ** : solo se consideran las componentes horizontal y vertical de la velocidad.
- Se **ignora la resistencia aerodinámica**, dado el tamaño del robot y las velocidades de salto relativamente bajas.
- La posición vertical del COM en el instante de despegue, Z_0 , es **constante y conocida** ($Z_0 = 0,029m$).

2.1.2. Tiro parabólico

Figura 2.1: Esquema de tiro parabólico



Las ecuaciones de movimiento en dos dimensiones, basándonos en 2.1 son:

$$x(t) = x_0 + v_0 \cos \theta \cdot t \quad (2.1)$$

$$z(t) = Z_0 + v_0 \sin \theta \cdot t - \frac{1}{2} g t^2 \quad (2.2)$$

donde v_0 es la velocidad de despegue del COM, θ es el ángulo de lanzamiento respecto a la horizontal, g es la aceleración gravitatoria y Z_0 es la altura del COM en el instante del despegue.

La condición de contorno es que en $x = D$ (distancia horizontal al obstáculo), $z \geq h$ (altura del obstáculo). Despejando el tiempo de vuelo de la ecuación (2.1) y sustituyendo en (2.2):

$$v_0 = D \sqrt{\frac{g}{2 \cos^2 \theta (Z_0 + D \tan \theta - h)}} \quad (2.3)$$

El objetivo es minimizar $v_0(\theta)$ para reducir la energía necesaria. **No existe solución analítica para el θ óptimo**, por lo que se realiza una búsqueda numérica por discretización del intervalo de ángulos válidos.

2.1.3. Restricción de rozamiento

Un ángulo de lanzamiento demasiado próximo a la horizontal haría que la componente horizontal de la fuerza del muelle, $F_x = F_{\text{muelle}} \cos \theta$, superase la fuerza de rozamiento estático entre el pie y el suelo, $F_r = \mu F_y$. En el caso más desfavorable (cuando la fuerza del muelle es mucho mayor que el peso estático), la condición de no patinamiento se simplifica a:

$$F_{\text{muelle}} \cos \theta \leq \mu F_{\text{muelle}} \sin \theta \implies \theta \geq \arctan\left(\frac{1}{\mu}\right) \quad (2.4)$$

Con un coeficiente de rozamiento de la goma $\mu = 0,90$, el ángulo mínimo de búsqueda es $\theta_{\min} = \arctan(1/0,90) \approx 48,0$.

2.1.4. Conservación de la energía

Una vez conocidos v_0 y θ , se calcula la compresión necesaria del muelle Δx igualando la energía potencial elástica almacenada (con eficiencia η) con la energía cinética y potencial gravitatoria en el momento del despegue. Como el COM se desplaza en diagonal una distancia Δx a lo largo del eje de la pata de salto, la variación de energía potencial gravitatoria es $Mg\Delta x \sin \theta$:

$$\eta \left(\frac{1}{2} k \Delta x^2 \right) = \frac{1}{2} M v_0^2 + Mg \Delta x \sin \theta \quad (2.5)$$

Resolviendo la ecuación de segundo grado en Δx :

$$\Delta x = \frac{Mg \sin \theta + \sqrt{M^2 g^2 \sin^2 \theta + \eta k M v_0^2}}{\eta k} \quad (2.6)$$

2.1.5. Adaptación dinámica de la altura de preparación

Una vez calculados los parámetros ideales de vuelo, surge un problema físico al intentar que el robot adopte la postura de disparo. Al comprimir el muelle una distancia Δx , la longitud efectiva de la pata de salto se acorta irremediabilmente. Si el chasis intentase mantener una altura estática predefinida mientras se inclina al ángulo θ_{opt} , la pata retraída quedaría suspendida en el aire, o peor aún, el chasis colisionaría contra el suelo al intentar compensar la cinemática inversa.

Para evitar colisiones y asegurar que el pie de salto se mantiene firmemente apoyado contra el suelo durante la carga, el software calcula una altura de preparación dinámica (Y_{prep}). Basándose en la trigonometría básica de la pata extendida (L_0), la altura del chasis se ajusta en tiempo real según la compresión exigida:

$$Y_{\text{prep}} = (L_0 - \Delta x) \sin(\theta_{\text{opt}}) \quad (2.7)$$

Adicionalmente, se impone por software una restricción de seguridad ($Y_{\text{prep}} \geq 0,02m$) para garantizar que, en saltos de máxima energía que requieran mucha retracción, la "barriga" del robot nunca llegue a raspar el suelo.

2.1.6. Cálculo del tiempo de compresión

Para que el sistema de salto funcione con precisión en lazo abierto, es fundamental traducir la elongación lineal requerida por el muelle (Δx) en un comando temporal exacto para el servomotor actuador (t). Este cálculo se aborda descomponiendo la cinemática de la transmisión desde el carrete hasta el eje del motor.

En primer lugar, la compresión lineal se relaciona directamente con el ángulo de giro del carrete (ϕ_c , en radianes) a través de su radio (r_c):

$$\phi_c = \frac{\Delta x}{r_c} \quad (2.8)$$

El carrete no está acoplado de forma directa al servomotor, sino que es accionado a través de una reductora de tres etapas. Conociendo el número de dientes de los engranajes conductores (Z_1, Z_3, Z_5) y los conducidos (Z_2, Z_4, Z_c), la relación de transmisión total (i) del tren de engranajes se define como el producto de sus etapas:

$$i = \frac{Z_2 \cdot Z_4 \cdot Z_c}{Z_1 \cdot Z_3 \cdot Z_5} \quad (2.9)$$

Por consiguiente, el desplazamiento angular total que debe ejecutar el eje del servomotor (ϕ_s) para lograr la compresión deseada en el extremo del hilo es:

$$\phi_s = \phi_c \cdot i = \frac{\Delta x \cdot Z_2 \cdot Z_4 \cdot Z_c}{r_c \cdot Z_1 \cdot Z_3 \cdot Z_5} \quad (2.10)$$

Asumiendo que el motor gira a una velocidad angular constante (ω_s) durante la fase de carga, el tiempo necesario para completar el movimiento se obtiene de la relación cinemática básica $t = \phi_s / \omega_s$. Según las especificaciones del fabricante del servomotor Feetech SCS0009, cuando se opera en modo de rotación continua (modo motor), la velocidad de salida es directamente proporcional al valor del comando de control enviado (C_{pwm}). Este parámetro se define en un rango de 10 bits (de 0 a 1023), por lo que la velocidad angular de trabajo se expresa en función de la velocidad máxima teórica (ω_{max}):

$$\omega_s = \omega_{max} \cdot \frac{C_{pwm}}{1023} \quad (2.11)$$

Sustituyendo las expresiones de ϕ_s y ω_s en la ecuación del tiempo, se obtiene la ecuación analítica final implementada en el código de control del robot:

$$t = \frac{\Delta x \cdot Z_2 \cdot Z_4 \cdot Z_c \cdot 1023}{r_c \cdot Z_1 \cdot Z_3 \cdot Z_5 \cdot \omega_{max} \cdot C_{pwm}} \quad (2.12)$$

Esta ecuación permite al microcontrolador calcular dinámicamente los segundos de activación del servomotor para cualquier altura de salto solicitada. En la implementación física, este valor se multiplica por mil para configurar el temporizador del ESP32, garantizando que el muelle se tense exactamente hasta la cota Δx requerida antes de detener el actuador.

2.1.7. Implementación numérica

El listado 2.1 muestra la función en C++ que implementa el cálculo de los parámetros del salto. La búsqueda numérica discretiza el intervalo $[\theta_{min}, \pi/2 - 0,01]$ en 1000 puntos y selecciona el que minimiza v_0^2 .

Listing 2.1: Cálculo numérico de los parámetros del salto

```

1 void Robot::calcularParametrosSalto(double h_objetivo)
2 {
3     // Rango de búsqueda del ángulo de lanzamiento
4     constexpr int RESOLUCION = 1000;
5     const double theta_min = std::atan(1.0 / MU_FRICCION);
6     const double theta_max = (M_PI / 2.0) - 0.01;
7     const double paso_theta = (theta_max - theta_min) / (RESOLUCION -
8         1);
9
10    double v0_sq_min = std::numeric_limits<double>::infinity();
11    double theta_opt = 0.0;
12
13    // Búsqueda del ángulo que minimiza la velocidad de despegue
14    // necesaria.
15    // Con d_min == d_max la búsqueda en d es trivial; se mantiene el
16    // doble
17    // bucle por compatibilidad con futuras variaciones de la distancia
18    // .
19    for (int i = 0; i < RESOLUCION; ++i) {
20        double theta = theta_min + i * paso_theta;
21        double cos2 = std::cos(theta) * std::cos(theta);
22        double tan_theta = std::tan(theta);
23
24        // El Z0 de despegue depende de la inclinación del robot con la
25        // pata extendida
26        double z0_takeoff = LONGITUD_PATA_SALTO * std::sin(theta);
27        double denom = z0_takeoff + DIST_SALTO * tan_theta -
28            h_objetivo;
29
30        if (denom <= 0.0) continue; // La trayectoria no alcanza la
31        altura
32
33        double v0_sq = (DIST_SALTO * DIST_SALTO * G) / (2.0 * cos2 *
34            denom);
35        if (v0_sq < v0_sq_min) {
36            v0_sq_min = v0_sq;
37            theta_opt = theta;
38        }
39    }
40
41    if (std::isinf(v0_sq_min)) {
42        RCLCPP_ERROR(this->get_logger(),

```

```

35         "Ninguna trayectoria alcanza h = %.2f m. Salto
           cancelado.", h_objetivo);
36     return;
37 }
38
39     double v0 = std::sqrt(v0_sq_min);
40
41     // Convertir a angulo de inclinacion del chasis (complementario al
       de lanzamiento)
42     theta_opt_deg_ = 90.0 - theta_opt * (180.0 / M_PI);
43
44     // Compresion necesaria del muelle
45     double m_g_sin = MASA_ROBOT * G * std::sin(theta_opt);
46     double delta_x = (m_g_sin + std::sqrt(m_g_sin*m_g_sin + 2*
       EFICIENCIA*(K_MUELLE_1 + K_MUELLE_2) * MASA_ROBOT * v0_sq_min))
       / (2*EFICIENCIA*(K_MUELLE_1 + K_MUELLE_2));
47
48     y_prep_salto_ = (LONGITUD_PATA_SALTO - delta_x) * std::sin(
       theta_opt);
49     if (y_prep_salto_ < 0.02) y_prep_salto_ = 0.02;
50
51     // --- Resultados de diagnostico ---
52     RCLCPP_INFO(this->get_logger(),
53         "SALTO: v0=%.2f m/s | incl=%.1f | dist=%.2f m |
           delta_x_muelle=%.4f m | Y_prep=%.3f m",
54         v0, theta_opt_deg_, DIST_SALTO, delta_x, y_prep_salto_);
55
56     if ((delta_x*100) >= 2.4)
57     {
58         RCLCPP_WARN(this->get_logger(), "Compresion requerida (%.1f cm)
           excede el muelle. Salto NO viable.", delta_x * 100.0);
59         movimiento_servo_ = 0.0;
60     } else {
61         // Tiempo de activacion del servo
62         double t_servo_s = (Z2*Z4*Z_CARRETE*1023*delta_x) / (MG_0_1023
           * Z1*Z3*Z5 * W_MAX_SERVO * R_CARRETE);
63         movimiento_servo_ = t_servo_s*1000;
64     }
65
66 }
67 }

```

2.2. Diseño y comparativa de los tres mecanismos de salto

2.2.1. Criterios de diseño comunes

La principal dificultad de dimensionar un mecanismo de salto para este proyecto es que el **momento de liberación debe ser variable**: a diferencia de mecanismos clásicos de leva o engranaje cortado, que tienen un Δx fijo de fábrica, aquí se necesita que Δx sea decidido por software en tiempo real. Esto impone restricciones serias sobre el tipo de mecanismo de retención elegible.

2.2.2. Mecanismo 1: tornillo sin fin

El primer mecanismo gira un tornillo sin fin impreso en 3D mediante el servomotor. Al final del tornillo hay un pequeño engranaje, y debajo de este, otro. Girando en sentido horario, ambos engranajes patinan, permitiendo que el tornillo comprima el muelle a través de un saliente. Al girar en sentido antihorario, los engranajes engranan y desacoplan el tornillo del compresor, liberando el muelle de forma instantánea.

La idea es ingeniosa porque usa el propio sentido de giro como señal de carga o disparo. Sin embargo, el tornillo sin fin es el tipo de mecanismo con menor eficiencia mecánica de todos los engranajes (típicamente del 20–40% en metálicos; en piezas impresas en 3D, aún menos), lo que obliga a disponer de un número de muelles en paralelo que no es viable. A esto se suma que la impresión 3D introduce una rugosidad superficial que provoca que el mecanismo se atasque con frecuencia. Por estas razones, **el mecanismo 1 se descarta sin llegar a probarse de manera exhaustiva**.

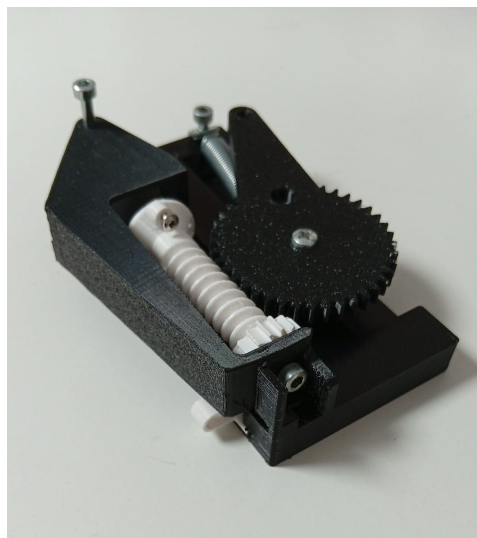


Figura 2.2: Mecanismo número 1. Implementación física. Planta



Figura 2.3: Mecanismo número 1. Implementación física. Perfil

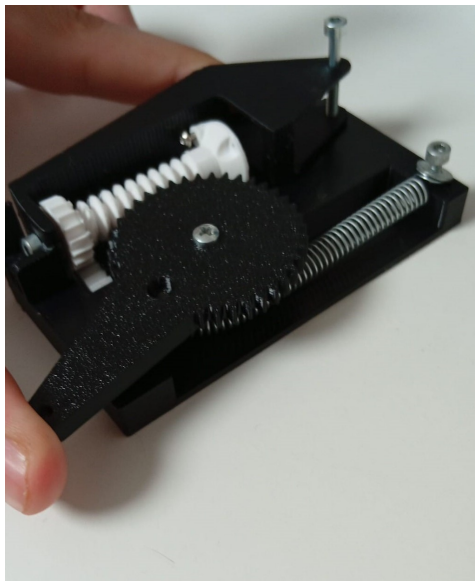


Figura 2.4: Mecanismo número 1. Implementación física. Carga del muelle

2.2.3. Mecanismo 2: trinquete y lengüeta con reductora

Este mecanismo es considerablemente más elaborado. Funciona sobre la base de un *ratchet and pawl*: el servomotor tiene acoplado un saliente perpendicular que, al girar, empuja tangencialmente los dientes de la rueda de trinquete, mientras la lengüeta impide el retroceso. Asociado a esto hay una reductora cuyo último engranaje lleva un carrete donde se enrolla un hilo. El hilo discurre por el interior de un tubo de plástico que actúa de “pata” (guía para que el muelle interior no flamee). En el extremo inferior hay un “pie” pasante que queda enganchado al hilo; al girar el mecanismo, el pie sube y comprime el muelle.

Para liberarlo, la lengüeta se diseñó con forma de U. Si el servomotor se desplaza más allá de su rango de compresión (por ejemplo, de 800–1023 a 400 PWM), el saliente choca con el brazo de la lengüeta, ejerciendo un momento que la hace rotar y libera toda la rueda de trinquete, disparando el muelle.

Figura 2.5: Mecanismo Ratchet And Pawl. SolidWorks

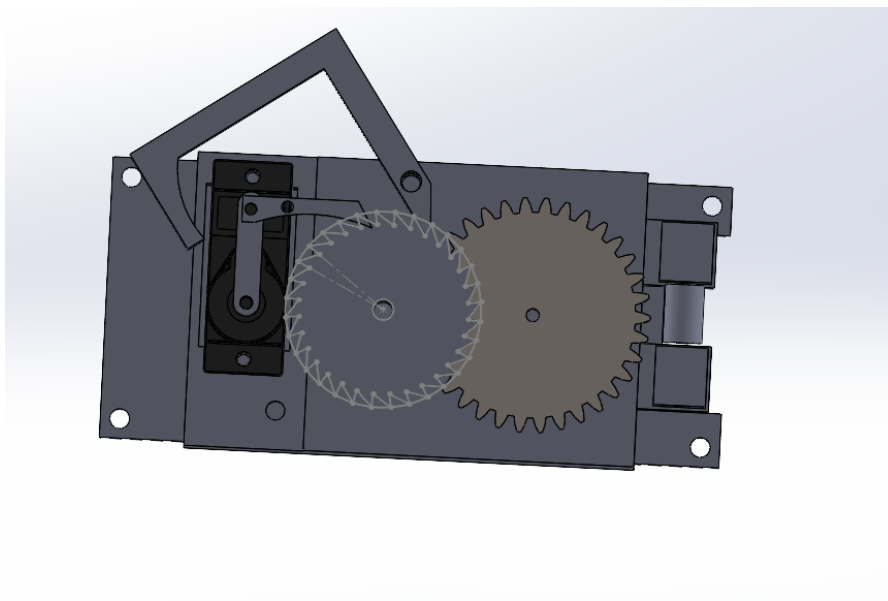
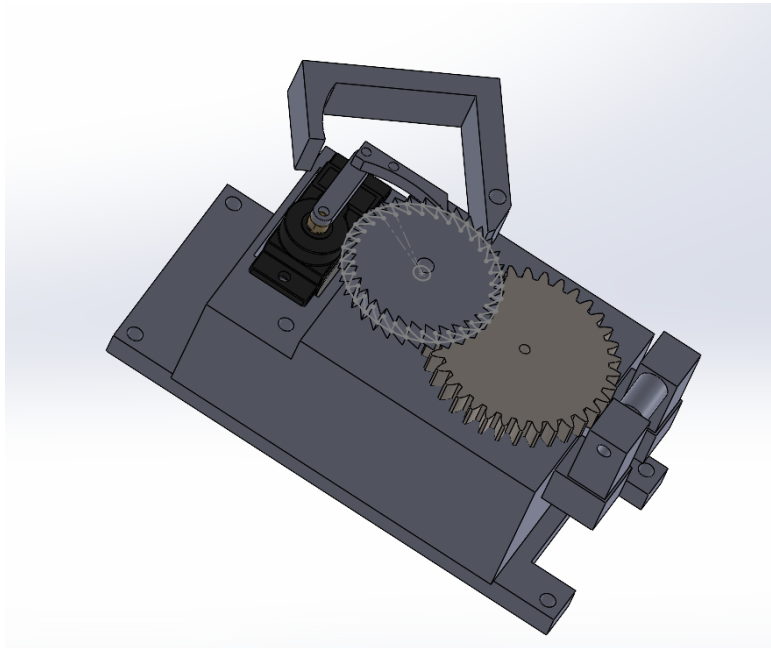


Figura 2.6: Mecanismo Ratchet And Pawl. SolidWorks. Vista desde Planta

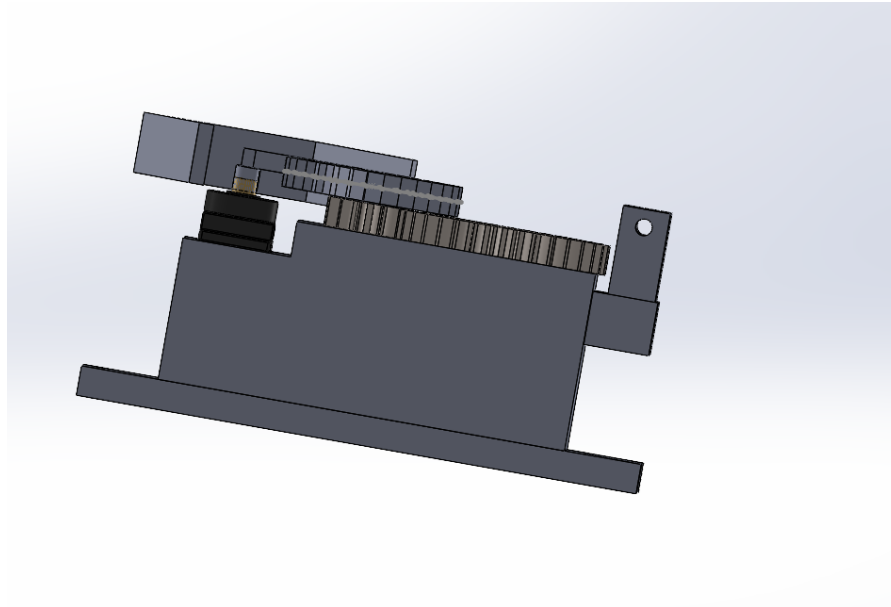


Figura 2.7: Mecanismo Ratchet And Pawl. SolidWorks. Vista desde Perfil

Ventajas:

- Solo requiere un servomotor para comprimir y disparar.
- El servo queda casi totalmente desacoplado del mecanismo durante la mayor parte del ciclo, lo que reduce la fatiga del actuador.

Inconvenientes:

- La resolución de la compresión es discreta: solo puede comprimirse en incrementos de $2\pi/n$ por diente, donde n es el número de dientes de la rueda.
- Solo puede usarse un muelle sin rediseñar el mecanismo por completo.
- En impresión 3D, la lengüeta y los dientes del trinquete se mellaban tras pocas decenas de ciclos.
- El pie tiende a atascarse en la guía porque la fuerza de compresión no es perfectamente vertical.

Dimensionamiento de la reductora: la relación de reducción necesaria viene dada por el cociente entre la fuerza máxima en el muelle ($k \cdot \Delta x_{\max}$) y el par del servomotor referido a la distancia de acción:

$$i = \frac{k \cdot (L_m \cdot 0,8)}{T_{\text{servo}} / d_{\text{servo}}} \quad (2.13)$$

Con $k = 620 \text{ N/m}$, $L_m = 7 \text{ cm}$, $T_{\text{servo}} = 73,55 \text{ mN} \cdot \text{m}$ y la geometría del carrete:

$$i \approx \frac{34,72}{1,84} \approx 18,9 \quad (2.14)$$

Se adopta una reductora con relación próxima a 20:1 para dejar margen de seguridad.

2.2.4. Mecanismo 3: doble muelle con disparo por desenganche directo (seleccionado)

Este es conceptualmente el más sencillo y el que mejores resultados ha dado. El servomotor compresor (ID 9, modo motor) gira la entrada de una reductora que tiene acoplado un carrete. Un hilo enrollado en el carrete tira simultáneamente de los extremos de dos muelles iguales en paralelo. Al mismo tiempo, la pata de salto romboidal adicional se retrae, acumulando la compresión.

El mecanismo de disparo se basa en un saliente móvil con un agujero situado en la reductora. Un alambre fino conecta ese agujero al brazo del segundo servo (ID 10, modo posición). Cuando se ordena el disparo, el servo 10 tira del alambre, arrastrando el saliente y desacoplando la reductora del carrete. El carrete queda libre y el muelle se extiende de forma instantánea, lanzando el robot.

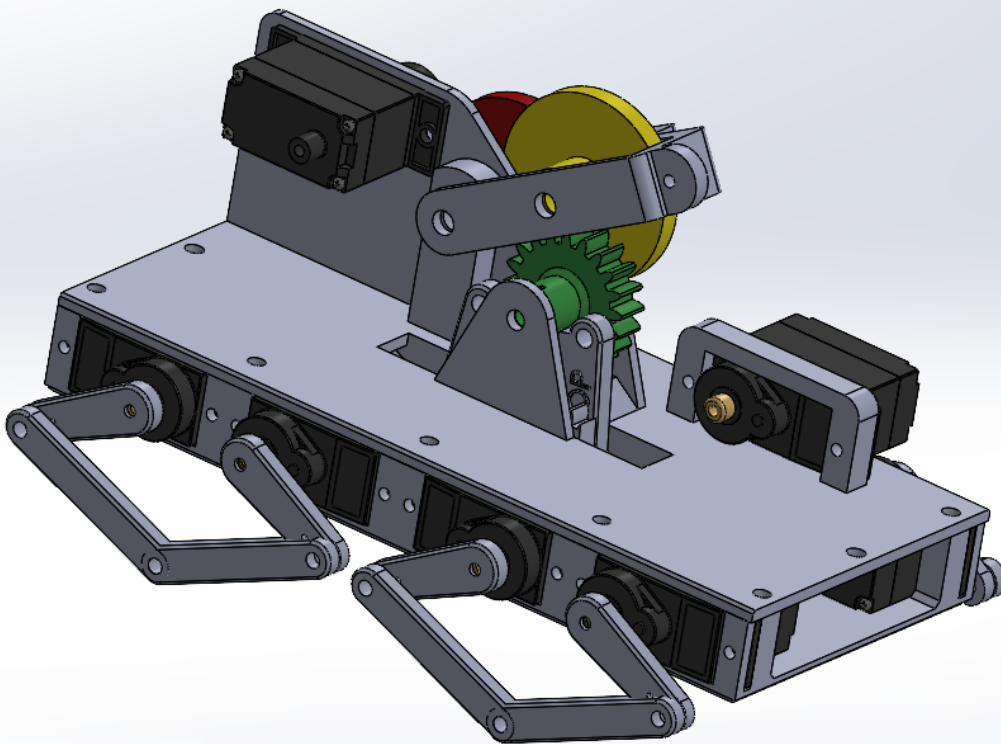


Figura 2.8: Ensamblaje del mecanismo 3. Solidworks.

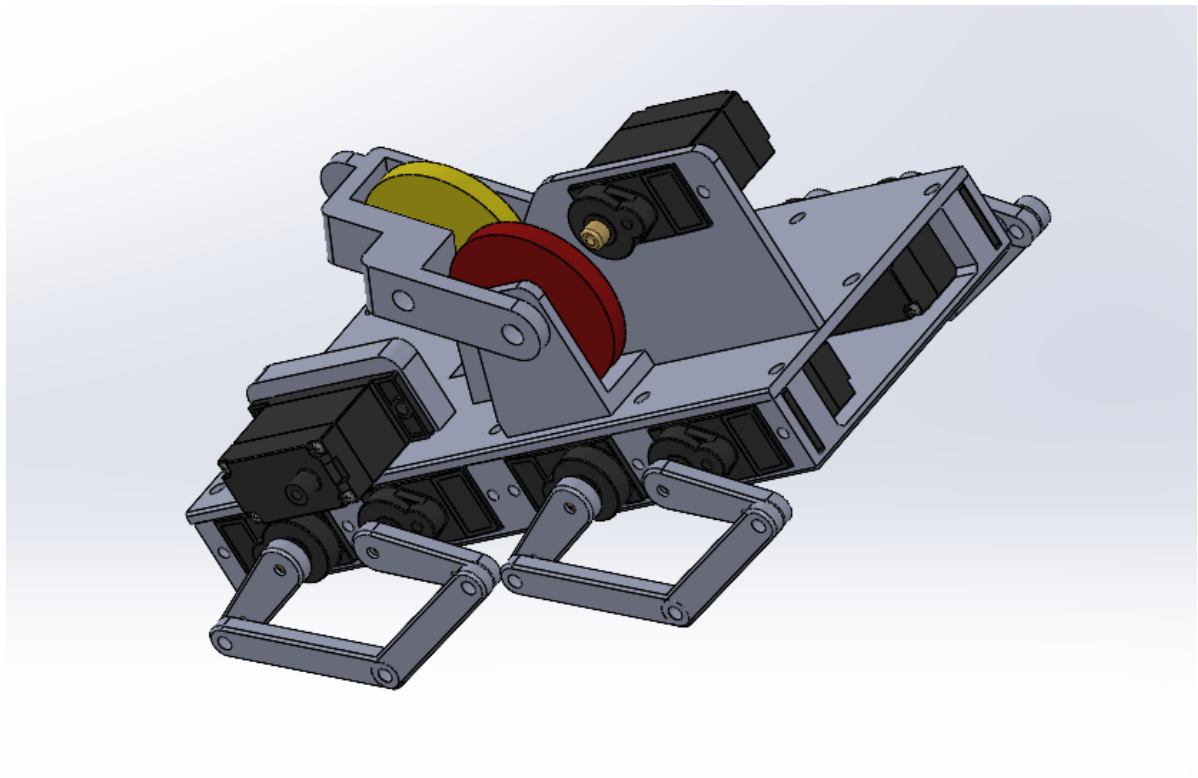


Figura 2.9: Ensamblaje del mecanismo 3. Solidworks. Parte trasera.

Ventajas:

- No hay dientes que se mellen ni fricciones extrañas: el funcionamiento es limpio.
- La estructura es modular: se pueden añadir más muelles en paralelo para aumentar la fuerza sin cambiar el diseño base.
- El par del servo se aprovecha al máximo porque actúa directamente sobre la entrada de la reductora sin distancias intermedias.
- La pata adicional de salto puede retraerse, permitiendo al robot agacharse más y aumentar el recorrido de aceleración.

Inconvenientes:

- Requiere dos servomotores (compresor y disparador).
- Entre salto y salto hay que re-tensar el hilo compresor manualmente (o con un ciclo de retroceso del servo 9).

Compresión requerida:

Si se recuerda la ecuación 2.5, aquí vamos a tener en cuenta que la energía potencial elástica viene dada por 4 muelles:

$$\eta \left(\frac{1}{2} k_1 \Delta x_1^2 \right) + \eta \left(\frac{1}{2} k_2 \Delta x_2^2 \right) + \eta \left(\frac{1}{2} k_3 \Delta x_3^2 \right) + \eta \left(\frac{1}{2} k_4 \Delta x_4^2 \right) = \frac{1}{2} M v_0^2 + M g \Delta x \sin \theta \quad (2.15)$$

Por el propio diseño del mecanismo, todas las compresiones son iguales, y 2 muelles son idénticos a los otros 2, por lo tanto:

$$\eta \Delta x^2 (k_1 + k_2) = \frac{1}{2} M v_0^2 + M g \Delta x \sin \theta \quad (2.16)$$

Por lo que la compresión requerida será:

$$\Delta x = \frac{M g \sin \theta + \sqrt{M^2 g^2 \sin^2 \theta + 2 \eta (k_1 + k_2) M v_0^2}}{2 \eta (k_1 + k_2)} \quad (2.17)$$

Dimensionamiento de la reductora:

Con dos muelles de tracción en paralelo, la fuerza máxima conjunta en el carrete es:

$$F_{\max} = 2 \cdot k \cdot (L_m \cdot 1,5) \cdot r_{\text{carrete}} \quad (2.18)$$

donde el factor 1,5 es la elongación máxima recomendada para muelles de tracción (relación longitud-deformación). La relación de reducción necesaria será:

$$i = \frac{F_{\max}}{T_{\text{servo}} / r_{\text{entrada}}} \quad (2.19)$$

Los valores concretos se actualizarán una vez se confirmen las constantes de los muelles definitivos.

2.3. Diseño del robot cuadrúpedo

2.3.1. Topología del robot

El robot elegido es un cuadrúpedo con cuatro patas romboidales más una pata adicional de salto. Las cuatro patas de locomoción permiten que el robot ande, se agache e incline el chasis para adoptar el ángulo de lanzamiento calculado. La pata de salto, situada en el eje de simetría trasero, es la que almacena y libera la energía del muelle.

Todos los componentes estructurales (chasis, patas, reductora, guías del muelle) se han diseñado en SolidWorks 2025 e impreso en PLA/PETG.

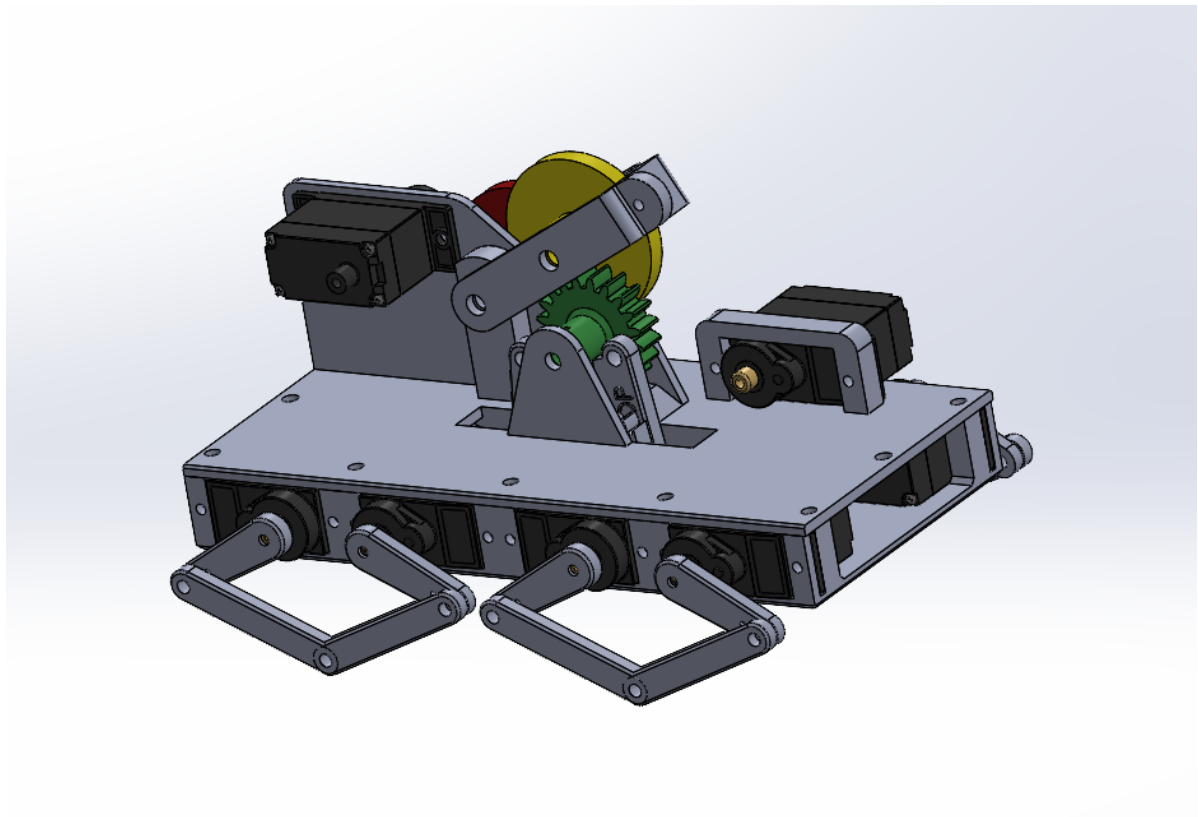


Figura 2.10: Ensamblaje completo. Solidworks.

2.3.2. Dimensionado de las patas

Las longitudes de los eslabones de las patas se dimensionaron para que los servos fuesen capaces de mover el robot, doblar las patas hasta el suelo y adoptar la posición de agachado necesaria antes del salto. Los valores geométricos finales se resumen en el cuadro 2.1.

Tabla 2.1: Parámetros geométricos de las patas

Parámetro	Símbolo	Valor
Separación entre pivotes	D_{pata}	22 mm
Eslabón superior (izq./der.)	$L_1 = L_2$	20 mm
Eslabón inferior (izq./der.)	$L_3 = L_4$	30 mm
Altura de reposo	Y_{reposo}	46 mm
Altura mínima (agachado)	Y_{agach}	20 mm

Esta geometría no solo responde a criterios cinemáticos, sino a una restricción estática crítica: el par nominal que pueden entregar los servomotores SCS0009, evaluado en $\tau_{\text{nom}} \approx 0,074 Nm$. Para asegurar que los motores no entren en pérdida (*stall*) durante la operación habitual, se plantea un modelo de estática de fuerzas en la posición más desfavorable. Esta situación se da cuando el robot

está muy agachado, momento en el cual la distancia horizontal efectiva desde la cadera hasta el punto de apoyo del pie (el brazo de palanca, L_{ext}) es máxima.

Asumiendo una extensión de palanca representativa para los cálculos de $L_{\text{ext}} = 0,05m$, el par necesario en la articulación depende de la fuerza vertical que soporta la pata (F_z). Se han evaluado dos escenarios limitantes para determinar la masa máxima admisible del robot (M_{max}):

Marcha en trote (2 patas en apoyo): En esta marcha dinámica, el peso del robot se reparte entre dos patas diagonales. Asumiendo que el mecanismo de cinco barras distribuye eficientemente la carga, la condición de equilibrio para que el servo pueda levantar el cuerpo es:

$$M_{\text{trote}} = \frac{2 \cdot \tau_{\text{nom}}}{g \cdot L_{\text{ext}}} \quad (2.20)$$

Apoyo sobre 1 pata: Es el peor caso posible, que puede darse ante una perturbación externa o durante un aterrizaje asimétrico tras un salto. Todo el peso recae sobre el brazo de palanca de una única pata:

$$M_{1\text{pata}} = \frac{\tau_{\text{nom}}}{g \cdot L_{\text{ext}}} \quad (2.21)$$

Evaluando estas ecuaciones, con el brazo de palanca elegido de 5,0 cm, la masa máxima que soportaría el robot a trote sin exceder el par nominal es de aproximadamente $2 \cdot 274 = 548g$ g en el peor de los casos, mientras que para sostenerse sobre una sola pata el límite se reduce a la mitad. Teniendo en cuenta que el peso mínimo de la electrónica y los 10 motores ya suma cerca de 140 g, estas longitudes de eslabón se confirman como un diseño equilibrado: mantienen la palanca lo suficientemente corta para no saturar los motores, pero lo suficientemente larga para proporcionar un buen rango de movimiento.

2.3.3. Selección final del elemento elástico

Finalmente, tomando el modelo teórico de salto desarrollado en la sección 2.1, se procedió a encontrar un muelle comercial que ofreciese el compromiso adecuado entre la constante elástica total (k) y el recorrido de compresión (Δx) necesario para alcanzar la cota máxima de diseño. Para este prototipo, se fijó como objetivo prioritario un salto límite de aproximadamente 6 cm ($h = 0,06$ m).

Introduciendo esta altura y la masa de 0,37 kg en la ecuación de conservación de la energía, y aplicando el factor de eficiencia calibrado de $\eta = 0,75$, el algoritmo matemático determina que la constante elástica ideal para el conjunto es $k \approx 350N/m$. Bajo esta configuración, las restricciones geométricas del chasis y el recorrido útil del carrete de tensado exigen una elongación lineal del muelle de unos 4,0 cm ($\Delta x \approx 0,040m$) en el instante previo al disparo.

Con estos parámetros definitivos, el sistema elástico almacenará una energía potencial de aproximadamente 0,27J en su máxima extensión. Al liberarse el gatillo, el muelle llegará a ejercer una fuerza lineal máxima de 13,9N. Este valor de fuerza es sumamente favorable, ya que garantiza que los componentes mecánicos no trabajarán al límite de su resistencia, permitiendo que tanto el pasador

de liberación como la estructura del chasis impresa en 3D absorban el impacto del disparo sin sufrir deformaciones plásticas ni fatiga prematura.

2.3.4. Cinemática inversa de las patas

La cinemática inversa de cada pata romboidal calcula los ángulos de los dos servos a partir de la posición cartesiana (x, y) deseada de la punta. El desarrollo sigue el formalismo presentado a continuación.

Para la pata de geometría simétrica romboidal, se definen los ángulos auxiliares:

$$\alpha_1 = \arctan\left(\frac{y}{x + D_{\text{pata}}/2}\right) \quad (2.22)$$

$$\alpha_2 = \arctan\left(\frac{y}{x - D_{\text{pata}}/2}\right) \quad (2.23)$$

y las distancias desde cada pivote a la punta:

$$a_1 = \sqrt{\left(x + \frac{D_{\text{pata}}}{2}\right)^2 + y^2} \quad (2.24)$$

$$a_2 = \sqrt{\left(x - \frac{D_{\text{pata}}}{2}\right)^2 + y^2} \quad (2.25)$$

Aplicando la ley del coseno:

$$b_1 = \frac{L_1^2 + a_1^2 - L_3^2}{2L_1a_1} \quad (2.26)$$

$$b_2 = \frac{L_2^2 + a_2^2 - L_4^2}{2L_2a_2} \quad (2.27)$$

Los ángulos de los servos en grados son:

$$\theta_1 = \alpha_1 + \arccos(b_1) \quad (2.28)$$

$$\theta_2 = \alpha_2 - \arccos(b_2) \quad (2.29)$$

Si $|b_1| > 1$ o $|b_2| > 1$, la posición solicitada está fuera del espacio de trabajo de la pata y la función devuelve sin modificar las posiciones de los servos. El listado 2.2 muestra la implementación en C++:

Listing 2.2: Cinemática inversa de una pata romboidal

```
1 void Robot::calcularPataIK(float x, float y, const Pata &p,
2                             int &posS1, int &posS2) const
3 {
```

```

4     float alfa1 = atan2(y, x + D_PATA / 2.0f);
5     float alfa2 = atan2(y, x - D_PATA / 2.0f);
6     float a1 = std::sqrt((x + D_PATA/2.0f)*(x + D_PATA/2.0f) + y*y);
7     float a2 = std::sqrt((x - D_PATA/2.0f)*(x - D_PATA/2.0f) + y*y);
8
9     float b1 = (L1*L1 + a1*a1 - L3*L3) / (2.0f * L1 * a1);
10    float b2 = (L2*L2 + a2*a2 - L4*L4) / (2.0f * L2 * a2);
11
12    if (std::abs(b1) > 1.0f || std::abs(b2) > 1.0f) return;
13
14    float theta1_deg = (alfa1 + std::acos(b1)) * (180.0f / M_PI);
15    float theta2_deg = (alfa2 - std::acos(b2)) * (180.0f / M_PI);
16
17    posS1 = gradosAPasos(p, 1, theta1_deg);
18    posS2 = gradosAPasos(p, 0, theta2_deg);
19 }

```

2.3.5. Inclinación del chasis para el salto

Para que el robot adopte el ángulo de lanzamiento θ , las patas delanteras deben bajar y las traseras mantener su posición. La variación de altura de la punta delantera respecto a la posición neutra es:

$$\Delta y_{\text{del}} = D_{\text{pata}} \cdot \tan \theta_{\text{rad}} \quad (2.30)$$

Esta corrección se aplica directamente en la función `publicarPoses`, que calcula la IK de las cuatro patas y publica los comandos al ESP32:

Listing 2.3: Publicación de poses con inclinación del chasis

```

1 void Robot::publicarPoses(float x_objetivo, float y_base,
2                           float theta_rad,
3                           int gatillo_s9, int gatillo_s10)
4 {
5     std_msgs::msg::Int32MultiArray msg;
6     msg.data.resize(10);
7
8     for (int i = 0; i < 4; i++) {
9         // Patas delanteras (índices 0 y 2): bajar segun inclinacion
10        float y_ik = (i == 0 || i == 2)
11                    ? y_base - D_PATA * std::tan(theta_rad)
12                    : y_base;
13
14        int s1, s2;

```

```

15         calcularPataIK(x_objetivo * misPatatas_[i].inversor_x,
16                        y_ik, misPatatas_[i], s1, s2);
17         msg.data[i*2]    = s1;
18         msg.data[i*2+1] = s2;
19     }
20     msg.data[8] = gatillo_s9;
21     msg.data[9] = gatillo_s10;
22     pub_poses_ ->publish(msg);
23 }

```

2.4. Arquitectura de software

2.4.1. Visión general

El sistema se divide en dos nodos ROS 2:

1. **Nodo nodo_robot** (PC, C++): contiene toda la lógica de control, la máquina de estados, la cinemática inversa y el cálculo de parámetros del salto. Publica los comandos de posición de los servos y los comandos al mecanismo de salto, y suscribe a los comandos de usuario y a la confirmación del ESP32.
2. **Nodo micro-ROS nodo_esp32_esclavo** (ESP32): recibe los arrays de posiciones de los servos y los reenvía al bus serie de Feetech. También gestiona el temporizador interno de activación del servo compresor y publica una confirmación cuando el tensado ha terminado.

La comunicación entre ambos nodos se realiza por WiFi mediante micro-ROS Agent en el PC.

2.4.2. Tópicos ROS 2

Tabla 2.2: Tópicos ROS 2 del sistema

Tópico	Tipo	Pub.	Sub.
servo_poses	Int32MultiArray	nodo_robot	ESP32
cmd_meta_salto	Int32	nodo_robot	ESP32
estado_salto_listo	Int32	ESP32	nodo_robot
cmd_vel	Twist	teleop	nodo_robot
cmd_estado	Int32	teleop	nodo_robot
cmd_salto	Float64	teleop	nodo_robot

2.4.3. Máquina de estados del nodo principal

El nodo principal implementa una máquina de estados finita con los estados del listado 2.4. El bucle principal corre a 50Hz (cada 20ms).

Listing 2.4: Enumeración de estados del robot

```

1  enum Estado {
2      ANDAR ,
3      AGACHADO ,
4      REPOSO ,
5      PREPARAR_SALTO ,
6      CARGAR_MUELLE ,
7      LIBERAR_MUELLE ,
8      RECUPERAR_REPOSO
9  };
    
```

La secuencia completa de un salto es la siguiente:

1. **REPOSO** → se recibe la orden de salto (altura h).
2. **CARGAR_MUELLE**: se calcula θ_{opt} y Δx , se envía al ESP32 el tiempo de tensado (*hardcodeado* actualmente), y el servo 9 gira en modo motor durante ese tiempo.
3. **REPOSO** (pausa de 1 s) → **PREPARAR_SALTO**: el ESP32 confirma que el muelle está cargado. El nodo principal inicia una transición suave de 2 s en la que inclina el chasis a θ_{opt} y baja el COM a Z_0 .
4. **LIBERAR_MUELLE**: se ordena al servo 10 que dispare. Se espera 0,5s para que el mecanismo actúe mecánicamente.
5. **RECUPERAR_REPOSO**: transición suave de 3 s de vuelta a la posición de reposo con $\theta = 0$.

2.4.4. Nodo esclavo ESP32

El ESP32 hace de intermediario entre ROS 2 y el bus serie de Feetech. Sus responsabilidades son:

- Suscribirse a `servo_poses` y reenviar cada posición al servo correspondiente por su ID.
- El servo ID 9 funciona en modo PWM continuo y tiene un *shield* software: cuando se está ejecutando el temporizador de tensado, se ignoran los comandos que lleguen por `servo_poses` para ese servo.
- Al recibir un mensaje en `cmd_meta_salto` con la duración en milisegundos, activa el servo 9 durante ese tiempo y después publica 1 en `estado_salto_listo`.

2.4.5. Script de teleoperación

El script `teleop_teclado.py` permite controlar el robot desde un terminal: W/S para avanzar/retróceder, A/D para girar, C para agacharse, J para iniciar la secuencia de salto y espacio para volver al reposo.

2.5. Proceso de montaje y calibración

2.5.1. Asignación de IDs a los servos

Antes del montaje, se asignaron IDs individuales a cada servo (1 al 10) usando el software de Feetech. Al servo 9 se le configuró en modo motor (ángulo máximo y mínimo iguales a 0) para que opere en PWM continuo. El resto permanecen en modo posición.

2.5.2. Calibración de rangos de los servos de las patas

De forma empírica con el software de Feetech, se obtuvieron los valores de PWM correspondientes a la posición horizontal y a la posición más baja de cada servo de cada pata, limitando además el rango para evitar colisiones mecánicas:

Listing 2.5: Configuración hardware de las patas

```
1 misPatas_[0] = { {1, 8, 299, 584, 941, 671}, 0.75f, 1.0f };
2 misPatas_[1] = { {3, 2, 715, 992, 890, 634}, 0.50f, 1.0f };
3 misPatas_[2] = { {7, 6, 183, 466, 907, 630}, 0.25f, -1.0f };
4 misPatas_[3] = { {4, 5, 650, 310, 475, 754}, 0.00f, 1.0f };
```

El primer par de enteros de cada pata es el ID de sus dos servos. Los cuatro siguientes son los PWM de posición horizontal y posición baja de cada servo. El `float` final es el desfase de fase en el ciclo de marcha (0 a 1) y el último es el inversor de la dirección X (+1.0f o -1.0f según la orientación de la pata).

2.5.3. Problemas encontrados durante el montaje

Durante las iteraciones de diseño y montaje se encontraron varios problemas relevantes:

- **Colisión de las patas con el chasis al retraerse:** en las primeras versiones, las patas delanteras colisionaban con la placa inferior del chasis al bajar al máximo. Se solucionó aumentando el espacio vertical en el chasis y rediseñando el perfil de la pata.
- **Pie atascado en la guía del muelle** (mecanismo 2): la fuerza de compresión tenía una componente lateral no nula que empujaba el pie contra las paredes de la guía. En el tercer mecanismo

se eliminó la guía tubular y se usaron dos muelles en paralelo con ancla directa, lo que resolvió el problema.

- **Deformación de la lengüeta del trinquete** (mecanismo 2): el PLA impreso no aguantaba la carga cíclica en los dientes de la rueda de trinquete y en la lengüeta. Se probó con PETG con mejora moderada. El tercer mecanismo evita completamente piezas sometidas a cargas de impacto cíclicas.
- **Tensión del hilo compresor:** el hilo de pesca trenzado pierde algo de tensión entre usos. Se añadió un paso de pre-tensado al inicio de cada ciclo de salto.
- **Cantidad de cableado:** Debido a la propia topología y diseño de los servomotores, tienen demasiado cableado asociado que impedía enormemente el salto. Para solucionarlo, se soldaron conectores tipo “Jumper” juntos.

Implementación

3.1. Enfoque de diseño

El desarrollo de plataformas robóticas que combinan mecánica, electrónica y software requiere una metodología de trabajo que permita gestionar la complejidad del sistema y adaptarse a los imprevistos. Por ello, en lugar de utilizar un enfoque lineal tradicional, en este proyecto se ha seguido un proceso de diseño mecatrónico iterativo [20]. Esta forma de trabajar permite evaluar rápidamente si las soluciones propuestas son viables en el mundo real antes de fabricar la versión final.

En concreto, la metodología empleada se divide en tres grandes etapas que se realimentan entre sí:

1. **Conceptualización:** identificación del problema, revisión de alternativas mecánicas y selección de componentes.
2. **Diseño y modelado:** modelado teórico del salto, diseño paramétrico del chasis y los mecanismos en SolidWorks 2025, e impresión 3D de prototipos.
3. **Implementación y validación:** montaje físico, programación y ensayos experimentales con retroalimentación para corregir fallos.

Gracias a este ciclo iterativo, los fallos detectados durante la fase de validación física (por ejemplo, fricciones inesperadas, ajustes en las tolerancias de la impresión 3D o problemas en la integración de los servomotores) sirven directamente para refinar el diseño paramétrico en el software CAD. Esto resulta fundamental en un proyecto orientado al bajo coste y la fabricación rápida, ya que minimiza el desperdicio de material y asegura que cada nueva versión del prototipo avance sobre seguro.

3.2. Lista de materiales (BOM) y presupuesto

Uno de los pilares fundamentales que motivaron este proyecto, tal y como se expuso en la introducción, es la democratización del acceso a plataformas robóticas ágiles. La robótica con patas múltipedas con capacidad de salto ha estado históricamente reservada a laboratorios de investigación con presupuestos abultados, utilizando actuadores de diseño propio (*custom quasi-direct drives*) y piezas mecanizadas en aluminio o fibra de carbono, elevando el coste de cada unidad a miles o decenas de miles de euros.

Para demostrar la viabilidad económica de la solución propuesta en este trabajo, se ha elaborado la lista de materiales final (*Bill of Materials* o BOM). En la tabla 3.1 se desglosa el coste directo de fabricación del prototipo, asumiendo la compra de componentes comerciales estándar y la fabricación propia de las piezas estructurales.

Tabla 3.1: Lista de materiales (BOM) y presupuesto de ejecución material del prototipo

Categoría	Componente	Cant.	Precio unit. (€)	Total (€)
3* Electrónica	Microcontrolador NodeMCU ESP32	1	6,50	6,50
	Tarjeta controladora de bus serie TTL	1	8,90	8,90
	Batería LiPo 2S 7,4V 1000mAh 25C	1	14,50	14,50
1* Actuadores	Servomotor bus serie Feetech SCS0009	10	12,00	120,00
2* Mecánica	Muelles de tracción ($k \approx 350$ N/m)	4	1,50	6,00
	Hilo trenzado de alta resistencia (Bobina)	1	5,00	5,00
3* Ferretería	Tornillería métrica variada (M2 y M3)	1	10,00	10,00
	Rodamientos miniatura (F623ZZ / MR63ZZ)	8	0,80	6,40
	Cableado, conectores y estaño	1	8,00	8,00
1* Estructural	Filamento PLA/PETG (aprox. 400g usados)	0,4	20,00/kg	8,00
COSTE TOTAL ESTIMADO DEL HARDWARE:				193,30 €

Como se puede observar en el desglose, el coste total de los materiales no supera los 200 €. Este hito se logra gracias a decisiones de diseño críticas:

1. **Uso de actuación elástica (LaMSA):** Al derivar la demanda de potencia de salto a la energía almacenada pasivamente en los muelles de tracción (6,00 €), se evita la necesidad de comprar motores de altísimo par y velocidad, permitiendo que todo el chasis se mueva exclusivamente con micro-servos comerciales de 12 €.
2. **Fabricación aditiva masiva:** Todas las piezas no electrónicas ni elásticas del robot (chasis, patas, eslabones, carrete y mecanismo de retención) han sido diseñadas para ser fabricadas

mediante impresión 3D FDM. Esto reduce el coste de mecanizado a prácticamente cero, requiriendo únicamente unos pocos gramos de filamento termoplástico.

3. **Electrónica de consumo y software libre:** La delegación de los cálculos de alto nivel a un PC externo y el uso de micro-ROS permiten gestionar un hardware tan complejo como este utilizando un microcontrolador ESP32 genérico, cuyo coste es inferior a 7 €.

En conclusión, el presupuesto confirma que la arquitectura desarrollada cumple con creces el objetivo de bajo coste, logrando replicar capacidades dinámicas complejas por una fracción del precio de las plataformas robóticas equivalentes del estado del arte.

3.3. Herramientas de software

- **SolidWorks 2025:** diseño de todas las piezas del chasis, patas y mecanismo de salto.
- **ROS 2 Humble** (Ubuntu 22.04): nodo de control principal en C++.
- **micro-ROS** sobre Arduino framework (ESP32): nodo esclavo embebido.
- **Feetech SCServo SDK:** biblioteca de bajo nivel para comunicación con los servos desde el ESP32.
- **Python 3:** script de teleoperación por teclado (`teleop_teclado.py`).

3.4. Montaje físico final

A continuación, se adjuntan fotos del ensamblaje final de todo el robot, con todas las correcciones mecánicas y componentes.

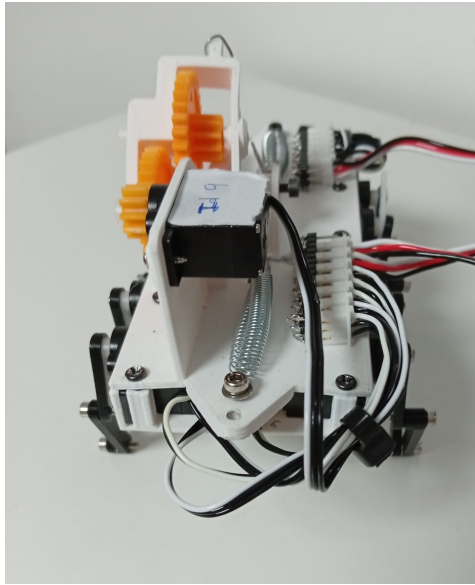


Figura 3.1: Montaje final. Vista de frente

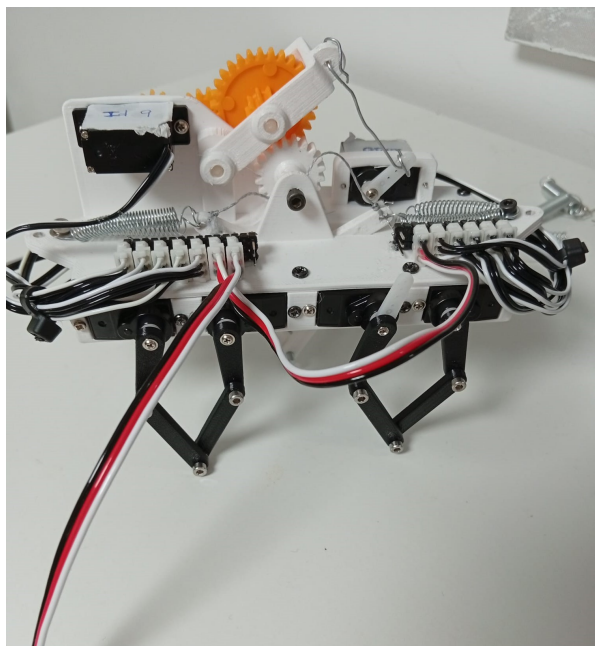


Figura 3.2: Montaje final. Vista de perfil

3.5. Criterios de validación

Los criterios para considerar válido un resultado experimental son:

1. El robot supera el obstáculo objetivo sin colisionar con él.
2. El robot mantiene el equilibrio tras el aterrizaje (o al menos no vuelca de forma violenta).

3. La compresión del muelle calculada teóricamente es coherente con el comportamiento observado (el robot no salta claramente más o menos de lo necesario).

Experimentación

Una vez implementado el prototipo físico y el modelo matemático de control, es necesario validar su rendimiento en un entorno real. El objetivo de este capítulo es doble: en primer lugar, calibrar los parámetros empíricos del sistema (específicamente la eficiencia de transferencia energética) y, en segundo lugar, evaluar la precisión del robot al ejecutar saltos con diferentes restricciones de altura y desplazamiento.

4.1. Metodología de medición

Para obtener datos cinemáticos precisos durante los breves instantes que dura la fase de vuelo, se ha utilizado el software de análisis de vídeo y fotogrametría 2D Kinovea. Los saltos se grabaron con una cámara de teléfono móvil normal, colocando marcadores visuales en el centro de masas (COM) del robot y estableciendo una referencia métrica en el plano de fondo para escalar las distancias de las capturas. De este modo, el software permite extraer frame a frame la trayectoria real y la altura máxima alcanzada.

Para garantizar el rigor de la validación, cada configuración de prueba documentada en este capítulo se ha repetido en tres ocasiones independientes. Los resultados numéricos presentados corresponden a la media de dichos ensayos, mitigando así las posibles variaciones provocadas por tolerancias mecánicas, diferencias milimétricas en la fricción del suelo o ligeras caídas de tensión en la batería.

4.2. Calibración del factor de eficiencia (η)

El modelo de conservación de energía desarrollado en el capítulo anterior depende de un parámetro crítico: la eficiencia del sistema (η). Si el modelo asume un sistema ideal ($\eta = 1$), considera que toda la energía elástica almacenada en el muelle se transforma íntegramente en energía cinética y potencial gravitatoria. Sin embargo, en la práctica física parte de esa energía se disipa inevitablemente debido al rozamiento en las articulaciones, la deformación elástica de las piezas impresas en 3D y las pérdidas por impacto en el mecanismo de disparo.

Para aislar este comportamiento y encontrar el valor real de η , se programó un salto vertical puro (sin desplazamiento horizontal) con una altura objetivo constante de 5 cm ($h = 0,05$ m) excepto para $\eta = 0,7$ que se fijó $h = 0,04$ m por existir la posibilidad de sobrepasar el límite de compresión máximo. Se introdujeron en el código tres coeficientes diferentes: $\eta = 1,0$, $\eta = 0,90$ y $\eta = 0,7$, permitiendo que el software calculara de forma autónoma la compresión de muelle necesaria para cada caso.

Los resultados obtenidos mediante el análisis fotogramétrico se resumen en la tabla 4.1.

Tabla 4.1: Resultados de la calibración de la eficiencia (η) para un salto objetivo de $h = 4 - 5$ cm (media de 3 ensayos)

Factor η configurado	Altura COM inicio salto	Altura real medida (COM)	Error absoluto
1,00	2,7 cm	6,91 cm	0,79 cm
0,90	2,6 cm	6,97 cm	0,63 cm
0,7	2,7 cm	7,00 cm	0,3 cm

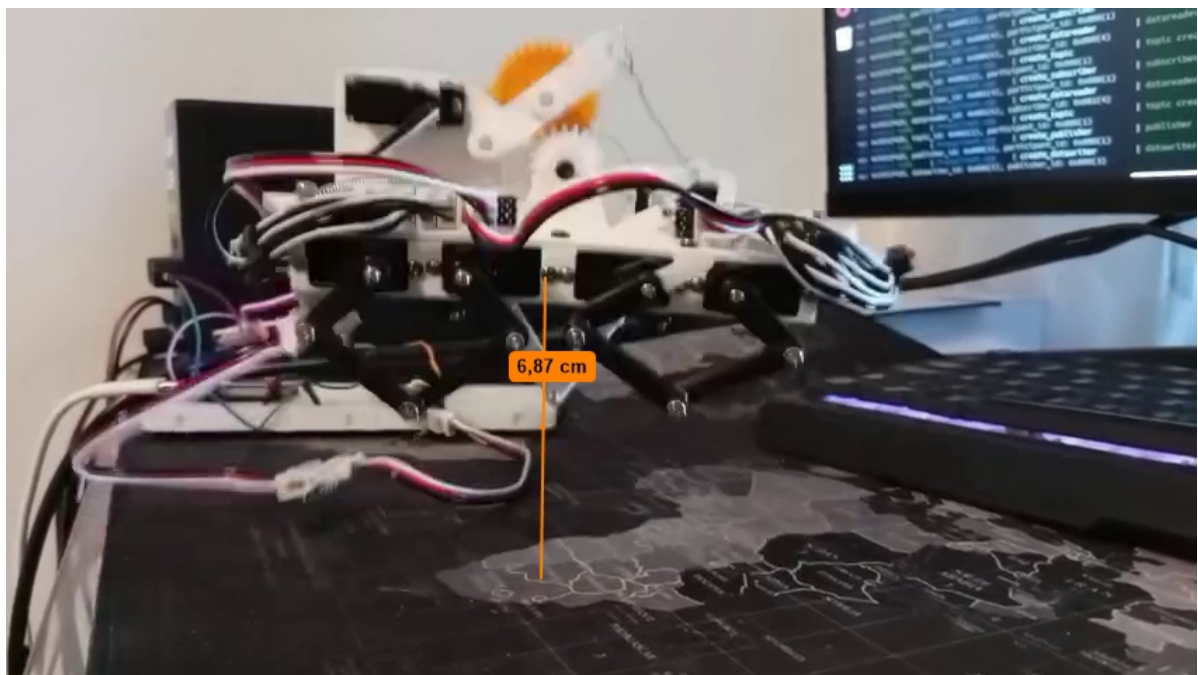


Figura 4.1: Experimento número 1. Fotograma de altura máxima. $\eta = 1$

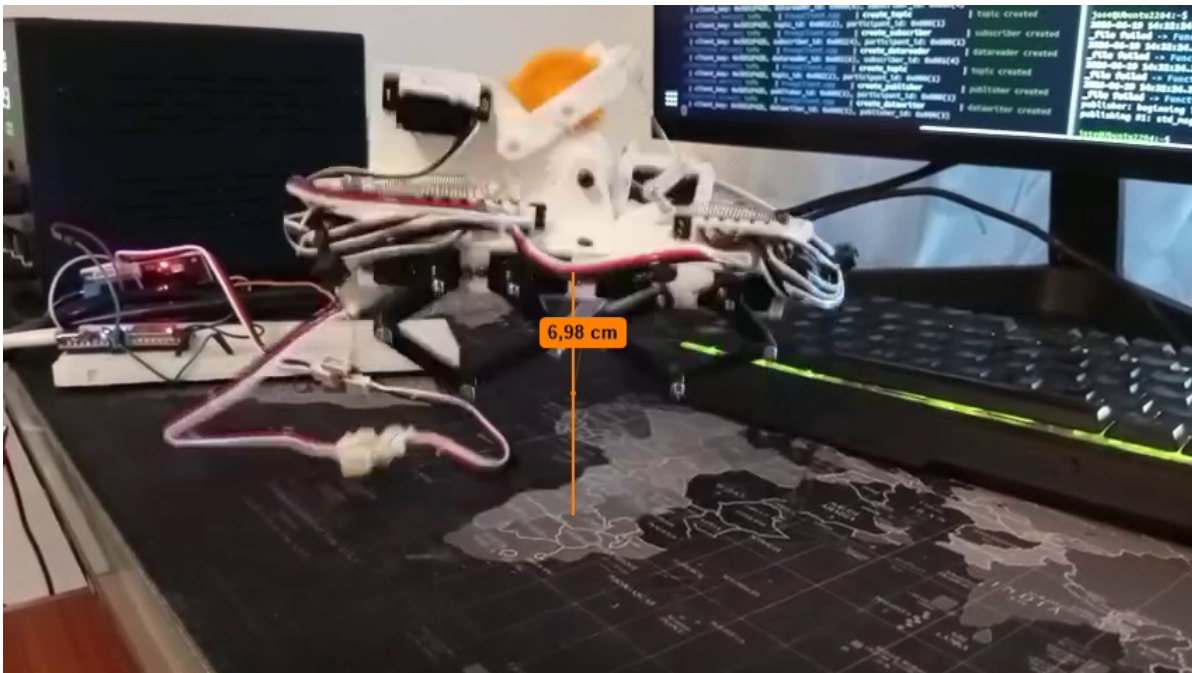


Figura 4.2: Experimento número 1. Fotograma de altura máxima. $\eta = 0,9$



Figura 4.3: Experimento número 2. Fotograma de altura máxima. $\eta = 0,7$

Como reflejan los datos, asumir una transferencia de energía perfecta ($\eta = 1,0$) provoca que el robot no alcance la altura demandada, ya que la compresión calculada es insuficiente para compensar las pérdidas mecánicas. Por el contrario, penalizar en exceso el sistema ($\eta = 0,70$, por ejemplo) hace que

el algoritmo sobredimensione el estiramiento del muelle, resultando en un salto que supera la cota de los 4 cm.

El coeficiente de $\eta = 0,75$ demostró ser el mejor ajuste empírico para esta plataforma mecánica, logrando que la altura real medida por Kinovea coincidiera con un margen de error mínimo respecto a la altura solicitada. A partir de este hallazgo, el parámetro se fijó de forma definitiva en un 75 % de eficiencia en el código fuente para acometer el resto de las pruebas de validación.

4.3. Validación del salto vertical a diferentes alturas

Con el factor de eficiencia fijado en $\eta = 0,75$ tras los ensayos de calibración, el siguiente paso es comprobar si el modelo matemático escala correctamente. Es decir, si al solicitar al robot distintas alturas, el sistema es capaz de adaptar la compresión del muelle de forma dinámica y alcanzar la meta con precisión. Para esta fase, se evaluaron dos escenarios de salto vertical puro: $h = 4$ cm y $h = 5$ cm.

4.3.1. Parámetros del sistema

Antes de analizar el rendimiento físico, conviene listar los parámetros mecánicos constantes que el microcontrolador utiliza para resolver las ecuaciones de salto. Estos valores, declarados en la cabecera del código de control, son la base sobre la que se calcula la trayectoria teórica y se resumen en la tabla 4.2.

Tabla 4.2: Parámetros físicos constantes utilizados en el algoritmo del robot

Parámetro	Variable en código	Valor	Unidades
Masa total del robot	MASA_ROBOT	0,370	kg
Constante elástica total	K_MUELLE_1 + 2	370	N/m
Eficiencia del sistema	EFICIENCIA	0,75	Adimensional
Aceleración gravitatoria	G	9,81	m/s ²
Long. de pata extendida	LONGITUD_PATA_SALTO	0,042	m

4.3.2. Resultados experimentales

Para cada altura objetivo, el algoritmo calcula la velocidad de despegue teórica (v_0) y la elongación lineal del muelle necesaria (Δx) para almacenar la energía requerida. Durante las pruebas en el banco de ensayo, se ejecutaron tres saltos consecutivos por cada altura. Los movimientos se grabaron con una cámara y se procesaron posteriormente en Kinovea rastreando la coordenada vertical del centro de masas (COM).

En la figura 4.4 se ilustra una de las capturas de fotogramas clave, donde se aprecia la altura alcanzada de forma experimental. Además, el material audiovisual íntegro de estos ensayos, incluyendo todas

las tomas realizadas, se ha subido a un repositorio complementario y puede visualizarse en el siguiente enlace: <https://drive.google.com/drive/folders/1-toTgJNn1QmbLloWvbszV815IB0qC5nD?usp=sharing>.

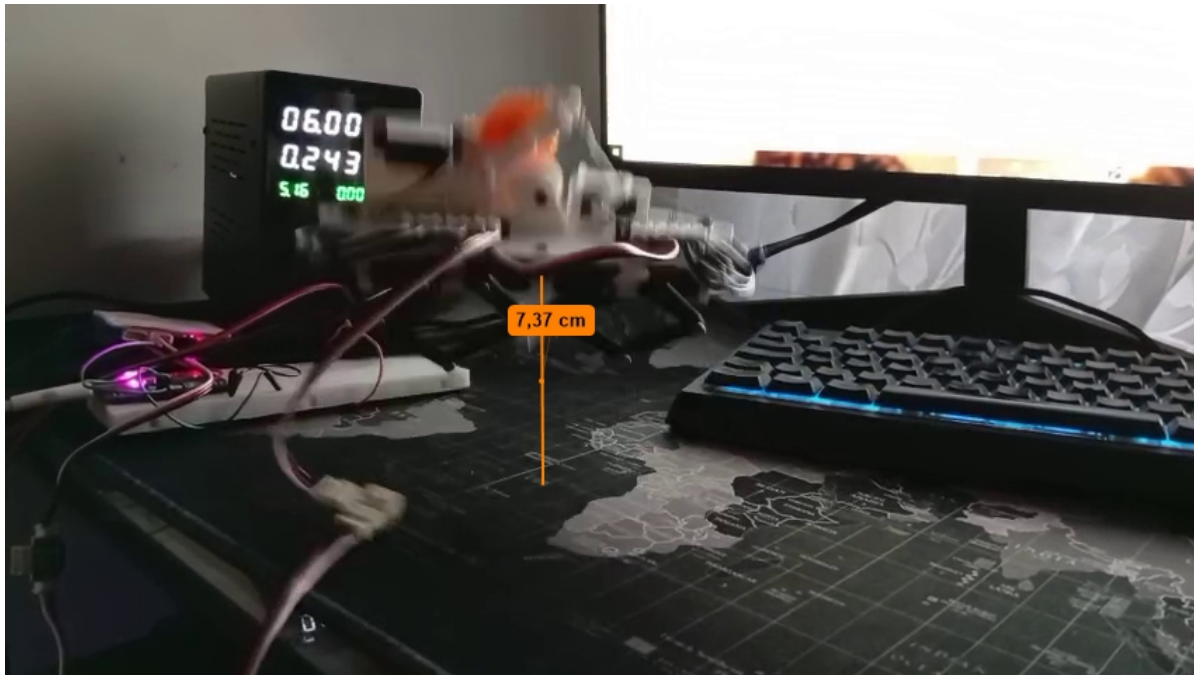


Figura 4.4: Análisis de la altura del COM con $h = 5cm$. Kinovea

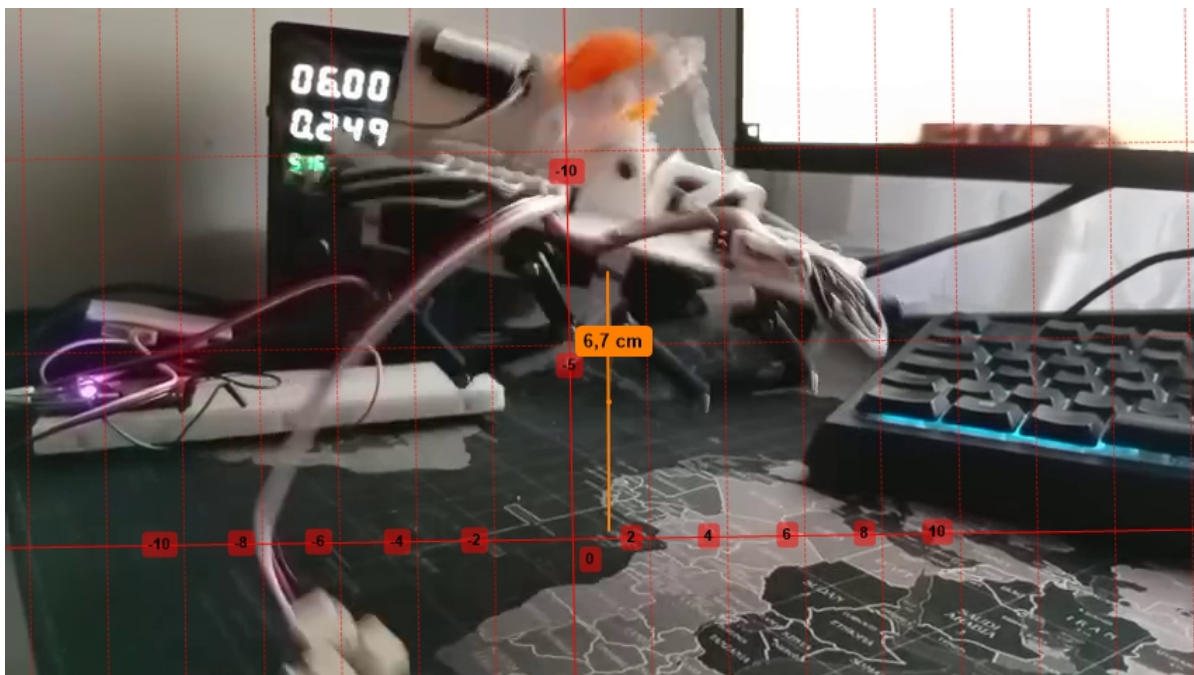


Figura 4.5: Análisis de la altura del COM con $h = 4cm$. Kinovea

Para aportar validez estadística al estudio, se extrajo la altura máxima alcanzada en cada uno de los intentos, calculando la media aritmética ($\bar{x}$) y la desviación típica (σ). La tabla 4.3 enfrenta los requerimientos teóricos dictados por el código con las mediciones empíricas recogidas en vídeo.

Tabla 4.3: Análisis cinemático y estadístico de los saltos verticales (media de 3 ensayos)

Meta (h)	Δx muelle	Altura real ($\bar{x}$)	Desviación (σ)	Distancia saltada	Error relativo
4,00 cm	1,34 cm	6,6845 cm	± 0.0153 cm	3,9845 cm	0,3875 %
5,00 cm	2,30 cm	7.3167 cm	± 0.0440 cm	5,0167 cm	0,3340 %

4.3.3. Análisis de precisión y fuentes de error

Como evidencian los datos, el cuadrúpedo es capaz de modular su energía de salto con una precisión altísima. Los errores relativos se sitúan en torno al 0,35 %, un margen sumamente estrecho tratándose de un sistema mecánico ensamblado con piezas impresas en plástico PLA, las cuales están sometidas inevitablemente a holguras, flexiones parasitarias y variaciones de fricción estática.

Resulta interesante observar la propagación del error en la desviación típica. Al demandar un salto de 5 cm, la dispersión de las medidas aumenta ligeramente ($\pm 0,0440$ cm frente a $\pm 0,0153$ cm en el salto de 4 cm). Físicamente, esta discrepancia tiene justificación: al estirar más el conjunto de muelles, la tensión máxima que soporta el chasis y el pasador del mecanismo es significativamente mayor. Al liberarse de golpe esta energía elástica, el impacto estructural genera micro-vibraciones en el cuerpo del robot milisegundos antes del despegue. Estas vibraciones provocan que el apoyo de los pies contra el suelo varíe ligeramente de un intento a otro, introduciendo un componente aleatorio en la eficiencia con la que se transfiere el momento lineal al suelo.

A pesar de estas pequeñas pérdidas cinéticas no modeladas matemáticamente (vibración estructural, ruido acústico y deformación transitoria del chasis), la cercanía de la media empírica a la meta teórica confirma de manera categórica que la resolución de la ecuación de energía implementada en el robot funciona correctamente en el mundo real.

Asimismo, durante los ensayos se observó que resulta extremadamente difícil lograr un ascenso puramente vertical. Esta desviación es consecuencia directa de la presencia del cableado de alimentación y telemetría. A pesar de haber procurado darle la máxima holgura para minimizar su influencia mecánica, en el ápice del salto el cable ejerce una ligera tensión. Esta pequeña fuerza externa genera un momento asimétrico respecto al centro de masas que acaba induciendo una rotación indeseada del robot en el aire.

Discusión y Conclusiones

5.1. Conclusiones

La experimentación de este trabajo de fin de grado ha demostrado que es completamente viable integrar un mecanismo de salto con altura controlable por software en un robot cuadrúpedo de bajo coste. Se ha logrado pasar de un concepto teórico a un prototipo físico funcional utilizando herramientas accesibles como la impresión 3D, servomotores de serie y el ecosistema de código abierto de ROS 2.

A partir del desarrollo, la implementación y las pruebas experimentales, se pueden extraer las siguientes conclusiones fundamentales:

1. **Evolución y selección del diseño mecánico:** El proceso iterativo de diseño fue clave para llegar a una solución funcional. De los tres mecanismos propuestos, el diseño basado en doble muelle en paralelo con desenganche directo demostró ser ampliamente superior. Los mecanismos alternativos (el tornillo sin fin y el trinquete con lengüeta) presentaron problemas inherentes a la fabricación en plástico, como fricciones excesivas, desgaste prematuro (*fatiga*) y la incapacidad de ofrecer una resolución de compresión continua. El mecanismo final, sin embargo, resultó ser limpio, modular y capaz de liberar la energía elástica de forma instantánea sin atascos.
2. **Validez del modelo teórico:** La aproximación matemática elegida, que combina la cinemática del tiro parabólico con la conservación de la energía elástica, ha demostrado ser sumamente precisa. Aunque se partía de simplificaciones fuertes (como tratar al robot como una partícula puntual y despreciar el rozamiento del aire), la inclusión de un factor de eficiencia empírico ($\eta = 0,75$) logró alinear la teoría con la realidad. Los experimentos validaron que el robot puede

clavar la altura demandada con un error relativo de apenas un 0,35 %, confirmando que la formulación es robusta y suficiente para este tamaño de plataformas.

3. **Rendimiento de la arquitectura de software:** La separación de las tareas de control ha sido un éxito. Utilizar un PC como nodo maestro ROS 2 permite resolver la cinemática inversa y las ecuaciones de salto en tiempo real y sin limitaciones de cálculo. Por su parte, delegar el control a bajo nivel de los servomotores a un microcontrolador ESP32 mediante micro-ROS garantiza que las órdenes físicas (como la temporización exacta del tensado del carrete) se ejecuten sin retardos ni bloqueos por culpa del sistema operativo principal.
4. **Control de postura mediante patas romboidales:** La formulación analítica de la cinemática inversa para la topología de cinco barras (rombo) no solo aligeró las patas, sino que permitió inclinar el chasis del robot al ángulo de lanzamiento exacto (θ_{opt}) de forma muy natural, adaptando la altura previa al salto para evitar colisiones con el suelo.

5.2. Limitaciones del trabajo

A pesar de los buenos resultados obtenidos en la validación, el prototipo actual actúa como un demostrador tecnológico y presenta ciertas limitaciones físicas y de control que deben tenerse en cuenta:

- **Dependencia del cableado (Tethering):** Como se evidenció en las pruebas con Kinovea, es extremadamente difícil lograr un salto vertical puro y estable. El bloque de cables de alimentación y telemetría introduce tensiones impredecibles en el ápice del salto. Esta fuerza descentrada genera un momento mecánico que altera la postura del robot en el aire, haciéndolo rotar.
- **Control en lazo abierto del muelle:** Actualmente, el sistema confía a ciegas en que el servomotor ha estirado el hilo exactamente la distancia Δx calculada. Al no existir un sensor que mida la extensión real del muelle, cualquier pequeña pérdida de tensión en el hilo de pesca trenzado o un leve deslizamiento en el carrete introduce un pequeño error en la energía acumulada.
- **Desgaste y durabilidad:** Las piezas impresas en PLA y PETG, aunque fáciles de sustituir, absorben picos de impacto considerables al frenar el mecanismo tras el disparo. Además, el hilo de tracción sufre rozamiento continuo, por lo que la durabilidad a largo plazo del sistema (miles de ciclos) no está garantizada sin labores de mantenimiento preventivo.
- **Dinámica post-despegue:** El modelo teórico se desentiende del robot una vez que los pies abandonan el suelo. Cualquier pequeña asimetría en el agarre de las patas de apoyo al empujar provoca que el robot se desvíe ligeramente de su plano bidimensional XZ .

5.3. Trabajo futuro

La plataforma desarrollada establece una base de hardware y software excelente sobre la que se pueden plantear numerosas líneas de investigación para futuros proyectos o trabajos de nivel de máster. Las mejoras más directas para profesionalizar el sistema son:

1. **Independencia física y energética (*Untethering*):** El paso más urgente es eliminar el cableado externo integrando la batería LiPo en el propio chasis (distribuyendo las masas para mantener el centro de gravedad equilibrado). La comunicación con el PC se realizaría exclusivamente mediante la red WiFi nativa del ESP32 gracias a micro-ROS, lo que resolvería el problema de las rotaciones indeseadas en el aire.
2. **Percepción del entorno y autonomía de decisión:** Actualmente, la geometría del salto (h y D) se introduce de forma manual. La integración de un pequeño sensor de distancia (como un radar ToF - *Time of Flight* o de ultrasonidos) permitiría al robot escanear el terreno, identificar la altura del escalón frente a él y ejecutar toda la secuencia matemática y de salto de forma autónoma.
3. **Sensórica y control de vuelo:** La inclusión de una Unidad de Medición Inercial (IMU) permitiría al robot conocer su estado de aceleración e inclinación. Con esta información, se podría cerrar el lazo de control durante la caída libre, moviendo las patas rápidamente en el aire para aprovechar la conservación del momento angular y garantizar que el chasis aterrice siempre nivelado y sobre los pies.
4. **Absorción de impactos (Aterrizaje activo):** Implementar terminales flexibles en las patas impresas en material elástico (TPU) y utilizar la lectura de corriente de los servos SCS0009 para generar un control de impedancia. De esta forma, al tocar el suelo, las extremidades podrían flexionarse suavemente simulando un comportamiento de resorte virtual para absorber la energía cinética de la caída sin dañar los engranajes mecánicos.
5. **Expansión a tres dimensiones:** Ampliar el modelo matemático para permitir que el robot calcule trayectorias no solo hacia adelante, sino diagonales o laterales, combinando la orientación de las patas caminantes con el mecanismo de disparo.



Código completo del nodo principal (robot.hpp)

Listing A.1: robot.hpp – definición de la clase Robot

```
1  #ifndef ROBOT_HPP
2  #define ROBOT_HPP
3
4  #include "rclcpp/rclcpp.hpp"
5  #include "std_msgs/msg/int32_multi_array.hpp"
6  #include "std_msgs/msg/int32.hpp"
7  #include "std_msgs/msg/float64.hpp"
8  #include "geometry_msgs/msg/twist.hpp"
9  #include <cmath>
10 #include <limits>
11 #include <algorithm>
12 #include <chrono>
13
14 using namespace std::chrono_literals;
15
16 // =====
17 // CONSTANTES CINEMATICAS DE LA PATA (metros)
18 // =====
19 constexpr float D_PATA = 0.022f; // Separacion entre los dos pivotes
    de la pata
```

```

20  constexpr float L1      = 0.020f;  // Longitud del eslabon superior
    izquierdo
21  constexpr float L2      = L1;      // Longitud del eslabon superior
    derecho (sim trico)
22  constexpr float L3      = 0.030f;  // Longitud del eslabon inferior
    izquierdo
23  constexpr float L4      = L3;      // Longitud del eslabon inferior
    derecho (sim trico)
24  constexpr double Z0     = 0.029;   // Altura de despegue del chasis (m)
25
26  // =====
27  // CONSTANTES DE MARCHA
28  // =====
29  constexpr float ALTURA_SUELO      = 0.040f;  // Altura nominal del
    chasis al suelo (m)
30  constexpr float ALTURA_LEVANT     = 0.015f;  // Altura que levanta la
    pata en el aire (m)
31  constexpr float Y_REPOSO           = 0.046f;  // Altura del chasis en
    posicion de reposo (m)
32  constexpr int    CICLO_MS           = 3000;    // Duracion de un ciclo
    de marcha completo (ms)
33  constexpr float FRACCION_AIRE      = 0.25f;    // Fraccion del ciclo que
    la pata pasa en el aire
34
35  // =====
36  // CONSTANTES DE MARCHA (servo de disparo)
37  // =====
38  constexpr int    SERVO_GATILLO_CERRADO = 800;    // PWM con el gatillo
    retenido
39  constexpr int    SERVO_GATILLO_ABIERTO = 1023;   // PWM con el muelle
    cargado/gatillo armado
40  constexpr int    SERVO_GATILLO_LIBRE   = 920;    // PWM al soltar (
    disparar) el muelle
41
42  // =====
43  // CONSTANTES FISICAS DEL MECANISMO DE SALTO
44  // =====
45  constexpr double MASA_ROBOT      = 0.370;    // Masa total del robot (kg)
46  constexpr double G               = 9.81;     // Gravedad (m/s^2)
47  constexpr double K_MUELLE_1      = 170.0;    // Constante elastica del
    muelle (N/m)
48  constexpr double K_MUELLE_2      = 200.0;    // Constante elastica del
    muelle (N/m)

```

```

49 constexpr double EFICIENCIA    = 0.75;    // Eficiencia mecanica del
    muelle
50 constexpr double MU_FRICCION    = 0.90;    // Coeficiente de friccion
    estatica
51 constexpr double DIST_SALTO     = 0.001;    // Distancia horizontal de
    despegue al obstaculo (m)
52 constexpr double ALT_OBSTAC_DEFAULT = 0.031; // Altura de obstaculo
    por defecto (m)
53 constexpr double LONGITUD_PATA_SALTO = 0.042;
54 constexpr int Z1 = 10;
55 constexpr int Z2 = 30;
56 constexpr int Z3 = 10;
57 constexpr int Z4 = 30;
58 constexpr int Z5 = 10;
59 constexpr int Z_CARRETE = 20;
60 constexpr int MG_0_1023 = 600;
61 constexpr double W_MAX_SERVO = 10.4719755;
62 constexpr double R_CARRETE = 0.004;
63
64
65 // Ciclos de pausa (a 50 Hz) al cambiar de estado
66 constexpr int CICLOS_PAUSA_ESTADO = 50; // 50    20 ms = 1 segundo
67
68 // =====
69 // ESTRUCTURAS DE DATOS
70 // =====
71
72 /// Par metros hardware de calibración de una pata (canales y
    posiciones PWM)
73 struct ConfigHardware {
74     int s1, s2;
75     int pwm_horiz_s1, pwm_down_s1;
76     int pwm_horiz_s2, pwm_down_s2;
77 };
78
79 /// Representa una pata con su configuración hardware y par metros de
    marcha
80 struct Pata {
81     ConfigHardware config;
82     float desfase;    // Desfase de fase en el ciclo de marcha [0, 1)
83     float inversor_x; // +1.0 o -1.0 seg n la orientación lateral de
        la pata
84 };

```

```

85
86 /// Estados de la m quina de estados del robot
87 enum Estado {
88     ANDAR,
89     AGACHADO,
90     REPOSO,
91     PREPARAR_SALTO,
92     CARGAR_MUELLE,
93     LIBERAR_MUELLE,
94     RECUPERAR_REPOSO
95 };
96
97 // =====
98 // CLASE ROBOT
99 // =====
100 class Robot : public rclcpp::Node
101 {
102 public:
103     Robot();
104     ~Robot();
105
106 private:
107     // --- Patas ---
108     Pata misPatas_[4];
109
110     // --- Maquina de estados ---
111     Estado estado_actual_ = REPOSO;
112     Estado estado_previo_ = REPOSO;
113     bool cambiado_ = false; // true cuando se acaba de cambiar
114                               de estado
115     int ciclos_espera_ = 0;
116
117     // --- Variables de locomoci n ---
118     float longitud_paso_ = 0.0f;
119     float compensacion_giro_ = 0.0f;
120     float altura_chasis_ = Y_REPOSO;
121     rclcpp::Time tiempo_inicio_; // Referencia temporal para la
122                                   marcha
123
124     // --- Variables de salto ---
125     double h_objetivo_ = ALT_OBSTAC_DEFAULT; // Altura del
126         obst culo (m)

```

```

124     double theta_opt_deg_      = 0.0;  //  ngulo    ptimo    de salto (
        grados)
125     float  theta_actual_      = 0.0f;  //  ngulo    de inclinaci n
        actual del chasis (grados)
126     int    movimiento_servo_  = 0;     //  Duraci n de tensado del
        servo (ms)
127     bool   orden_salto_enviada_ = false;
128     double y_prep_salto_ = 0.0;
129
130     // --- Fase interna del estado LIBERAR_MUELLE ---
131     int     fase_liberacion_    = 0;
132     bool    timer_liberacion_set_ = false;
133     bool    timer_disparo_set_  = false;
134     rclcpp::Time tiempo_fase_;
135
136     // --- Control de transiciones temporales (Interpolaci n) ---
137     bool    transicion_activa_ = false;
138     double  duracion_transicion_ = 0.0;
139     rclcpp::Time tiempo_inicio_transicion_;
140     float   y_inicial_ = 0.0f, y_final_ = 0.0f;
141     float   theta_inicial_ = 0.0f, theta_final_ = 0.0f;
142
143     // M todos para gestionar las transiciones de tiempo
144     void iniciarTransicion(float y_fin, float theta_fin, double
        duracion_segundos);
145     bool actualizarTransicion(float &out_y, float &out_theta);
146
147     // Nuevo estado
148     void estado_recuperar_reposo();
149
150     // --- ROS: Publishers ---
151     rclcpp::Publisher<std_msgs::msg::Int32MultiArray>::SharedPtr
        pub_poses_;
152     rclcpp::Publisher<std_msgs::msg::Int32>::SharedPtr
        pub_meta_salto_;
153
154     // --- ROS: Subscriptores ---
155     rclcpp::Subscription<geometry_msgs::msg::Twist>::SharedPtr
        sub_vel_;
156     rclcpp::Subscription<std_msgs::msg::Int32>::SharedPtr
        sub_estado_;
157     rclcpp::Subscription<std_msgs::msg::Float64>::SharedPtr
        sub_salto_;
    
```

```

158     rclcpp::Subscription<std_msgs::msg::Int32>::SharedPtr
        sub_salto_listo_;
159
160     // --- ROS: Timer principal ---
161     rclcpp::TimerBase::SharedPtr timer_control_;
162
163     // -----
164     // Callbacks ROS
165     // -----
166     void bucle_control_callback();
167     void cmd_vel_callback (const geometry_msgs::msg::Twist::
        SharedPtr msg);
168     void cmd_estado_callback (const std_msgs::msg::Int32::SharedPtr
        msg);
169     void cmd_salto_callback (const std_msgs::msg::Float64::SharedPtr
        msg);
170     void salto_listo_callback(const std_msgs::msg::Int32::SharedPtr
        msg);
171
172     // -----
173     // Lógica de estados
174     // -----
175     void estado_andar();
176     void estado_agachado();
177     void estado_reposo();
178     void estado_cargar_muelle();
179     void estado_preparar_salto();
180     void estado_liberar_muelle();
181
182     // -----
183     // Helpers de cinemática y trayectoria
184     // -----
185
186     /// Calcula las posiciones PWM de los dos servos de una pata
        mediante IK
187     void calcularPataIK(float x, float y, const Pata &p, int &posS1,
        int &posS2) const;
188
189     /// Rellena el array de 10 poses con las 4 patas en la posición IK
        (x, y_i)
190     /// permitiendo offsets de inclinación por pata. gatillo_s9 y
        gatillo_s10
191     /// se escriben en las posiciones [8] y [9] del array.

```

```

192     void publicarPoses(float x_objetivo, float y_base,
193                       float theta_rad,
194                       int gatillo_s9, int gatillo_s10);
195
196     /// Convierte ngulo en grados a posici n PWM para un servo de la
197     pata
198     int gradosAPasos(const Pata &p, int lado, float grados) const;
199
200     /// Devuelve la posici n (x, y) de la pata en el instante t_local
201     [0,1)
202     void calcularTrayectoriaPaso(float t_local, float longitud,
203                                  float &out_x, float &out_y) const;
204
205     /// Calcula los par metros ptimos del salto para superar
206     h_objetivo
207     void calcularParametrosSalto(double h_objetivo);
208
209     /// Prepara las variables de salto y transiciona a CARGAR_MUELLE
210     void iniciarSecuenciaSalto(double h);
211 };
212
213 #endif // ROBOT_HPP

```


Código principal del nodo Robot (robot.cpp)

Listing B.1: robot.cpp – definición e implementación de la lógica Robot

```

1  #include "tfg_salto/robot.hpp"
2
3  // =====
4  // CONSTRUCTOR / DESTRUCTOR
5  // =====
6
7  Robot::Robot() : Node("nodo_robot")
8  {
9      // --- Inicializar patas: {canal_s1, canal_s2, pwm_h_s1, pwm_d_s1,
10         pwm_h_s2, pwm_d_s2}, desfase, inversor_x ---
11      misPatas_[0] = { {1, 8, 299, 584, 941, 671}, 0.75f, 1.0f };
12      misPatas_[1] = { {3, 2, 715, 992, 890, 634}, 0.50f, 1.0f };
13      misPatas_[2] = { {7, 6, 183, 466, 907, 630}, 0.25f, -1.0f };
14      misPatas_[3] = { {4, 5, 650, 310, 475, 754}, 0.00f, 1.0f };
15
16      tiempo_inicio_ = this->now();
17
18      // --- Publishers ---
19      pub_poses_ = this->create_publisher<std_msgs::msg::
20         Int32MultiArray>("servo_poses", 10);

```

```

19     pub_meta_salto_ = this->create_publisher<std_msgs::msg::Int32>
        ("cmd_meta_salto", 10);
20
21     // --- Subscriptores ---
22     sub_vel_ = this->create_subscription<geometry_msgs::msg::Twist>(
23         "cmd_vel", 10, std::bind(&Robot::cmd_vel_callback, this, std::
        placeholders::_1));
24
25     sub_estado_ = this->create_subscription<std_msgs::msg::Int32>(
26         "cmd_estado", 10, std::bind(&Robot::cmd_estado_callback, this,
        std::placeholders::_1));
27
28     sub_salto_ = this->create_subscription<std_msgs::msg::Float64>(
29         "cmd_salto", 10, std::bind(&Robot::cmd_salto_callback, this,
        std::placeholders::_1));
30
31     sub_salto_listo_ = this->create_subscription<std_msgs::msg::Int32>(
32         "estado_salto_listo", 10, std::bind(&Robot::
        salto_listo_callback, this, std::placeholders::_1));
33
34     // --- Timer principal: 50 Hz (20 ms) ---
35     timer_control_ = this->create_wall_timer(
36         20ms, std::bind(&Robot::bucle_control_callback, this));
37
38     RCLCPP_INFO(this->get_logger(), "Nodo Robot iniciado.");
39 }
40
41 Robot::~Robot()
42 {
43     RCLCPP_INFO(this->get_logger(), "Apagando nodo del robot.");
44 }
45
46 // =====
47 // BUCLE PRINCIPAL (50 Hz)
48 // =====
49
50 void Robot::bucle_control_callback()
51 {
52     // Al cambiar de estado, se hace una pausa de 1 s en REPOSO antes
        de continuar
53     if (cambiado_) {
54         estado_previo_ = estado_actual_;
55         estado_actual_ = REPOSO;

```

```

56     ciclos_espera_ = CICLOS_PAUSA_ESTADO;
57     cambiado_ = false;
58 }
59
60     switch (estado_actual_) {
61         case ANDAR:          estado_andar();          break;
62         case AGACHADO:       estado_agachado();        break;
63         case REPOSO:         estado_reposo();          break;
64         case PREPARAR_SALTO: estado_preparar_salto();  break;
65         case CARGAR_MUELLE:  estado_cargar_muelle();  break;
66         case LIBERAR_MUELLE: estado_liberar_muelle();  break;
67         case RECUPERAR_REPOSO: estado_recuperar_reposo(); break;
68     }
69 }
70
71 // =====
72 // L GICA DE ESTADOS
73 // =====
74
75 void Robot::estado_andar()
76 {
77     // Tiempo global normalizado [0, 1) dentro del ciclo de marcha
78     auto tiempo_actual = this->now() - tiempo_inicio_;
79     double ms = tiempo_actual.nanoseconds() / 1e6;
80     float t_global = static_cast<float>(fmod(ms, CICLO_MS)) / CICLO_MS;
81
82     std_msgs::msg::Int32MultiArray msg;
83     msg.data.resize(10);
84
85     for (int i = 0; i < 4; i++) {
86         // Cada pata tiene su desfase de fase en el ciclo
87         float t_local = fmod(t_global + misPatas_[i].desfase, 1.0f);
88
89         // Las patas delanteras (0,1) restan la compensación y las
90         // traseras (2,3) la suman
91         float longitud = longitud_paso_ + ((i < 2) ? -
92             compensacion_giro_ : compensacion_giro_);
93
94         float x, y;
95         calcularTrayectoriaPaso(t_local, longitud, x, y);
96
97         int s1, s2;

```

```

96         calcularPataIK(x * misPatas_[i].inversor_x, y, misPatas_[i], s1
97             , s2);
98         msg.data[i*2] = s1;
99         msg.data[i*2+1] = s2;
100     }
101     msg.data[8] = 930; // Ratchet en posición de
102     marcha
103     msg.data[9] = SERVO_GATILLO_CERRADO; // Gatillo cerrado durante la
104     marcha
105     pub_poses->publish(msg);
106 }
107
108 void Robot::estado_agachado()
109 {
110     // Baja suavemente el chasis hasta y = 0.02 m (máximo agachado)
111     constexpr float Y_AGACHADO = 0.02f;
112     constexpr float PASO_BAJA = 0.0005f;
113
114     if (altura_chasis_ > Y_AGACHADO) {
115         altura_chasis_ = std::max(Y_AGACHADO, altura_chasis_ -
116             PASO_BAJA);
117     }
118
119     publicarPoses(0.0f, altura_chasis_, 0.0f, 0, SERVO_GATILLO_CERRADO)
120     ;
121 }
122
123 void Robot::estado_reposo()
124 {
125     constexpr float PASO_ALTURA = 0.0005f;
126     constexpr float PASO_THETA = 0.5f; // Grados por ciclo al
127     recuperar inclinación
128
129     // Recuperar altura suavemente hacia Y_REPOS0
130     if (std::abs(altura_chasis_ - Y_REPOS0) > PASO_ALTURA) {
131         altura_chasis_ += (Y_REPOS0 > altura_chasis_) ? PASO_ALTURA : -
132             PASO_ALTURA;
133     } else {
134         altura_chasis_ = Y_REPOS0;
135     }
136
137     // Recuperar inclinación suavemente hacia 0

```

```

132     theta_actual_ = std::max(0.0f, theta_actual_ - PASO_THETA);
133
134     // Si el estado previo era PREPARAR_SALTO, el muelle sigue cargado:
135     // no soltar el gatillo
136     int gatillo = (estado_previo_ == PREPARAR_SALTO) ?
137         SERVO_GATILLO_ABIERTO : SERVO_GATILLO_CERRADO;
138
139     publicarPoses(0.0f, altura_chasis_, theta_actual_ * (M_PI / 180.0f)
140         , 0, gatillo);
141
142     // Volver al estado anterior tras la pausa
143     if (ciclos_espera_ > 0) {
144         ciclos_espera_--;
145     } else {
146         estado_actual_ = estado_previo_;
147     }
148 }
149
150 void Robot::estado_cargar_muelle()
151 {
152     // Mantener las patas r gidas en reposo con el gatillo armado
153     publicarPoses(0.0f, altura_chasis_, 0.0f, 0, SERVO_GATILLO_ABIERTO)
154     ;
155
156     // Enviar la orden de tensado al ESP32 una sola vez
157     if (!orden_salto_enviada_)
158     {
159         std_msgs::msg::Int32 msg;
160         msg.data = movimiento_servo_;
161         pub_meta_salto_->publish(msg);
162         RCLCPP_INFO(this->get_logger(), "Orden de tensado al ESP32: %d
163             ms", movimiento_servo_);
164         orden_salto_enviada_ = true;
165     }
166 }
167
168 void Robot::estado_preparar_salto()
169 {
170     // Fase 1: Inclina el chasis y adapta la altura a la longitud
171     // sica de la pata contra da
172     if (!transicion_activa_ && !timer_liberacion_set_)
173     {

```

```

168         RCLCPP_INFO(this->get_logger(), "Ajustando postura a %.1f y
           %.3f m de altura...", theta_opt_deg_, y_prep_salto_);
169
170         // Aplicamos la altura dinámica (y_prep_salto_) en lugar del
           Z0 estático
171         iniciarTransicion(static_cast<float>(y_prep_salto_),
           static_cast<float>(theta_opt_deg_), 2.5);
172         timer_liberacion_set_ = true;
173     }
174
175     if (actualizarTransicion(altura_chasis_, theta_actual_))
176     {
177         estado_actual_ = LIBERAR_MUELLE;
178         fase_liberacion_ = 0;
179         timer_disparo_set_ = false;
180     }
181
182     publicarPoses(0.0f, altura_chasis_, theta_actual_ * (M_PI / 180.0f)
           , 0, SERVO_GATILLO_ABIERTO);
183 }
184
185 void Robot::estado_liberar_muelle()
186 {
187     // Fase 2: Liberar el mecanismo (ID 10 a 800)
188     if (fase_liberacion_ == 0) {
189         RCLCPP_INFO(this->get_logger(), " SALTO ! Liberando mecanismo
           ID 10 a 800...");
190
191         // gatillo_s10 ( índice 9 en tu array) lo enviamos a 800
192         publicarPoses(0.0f, altura_chasis_, theta_actual_ * (M_PI /
           180.0f), 0, 800);
193
194         tiempo_fase_ = this->now();
195         fase_liberacion_ = 1;
196     }
197     else {
198         // Mantén la publicación del gatillo suelto para asegurarnos
           de que el servo llega
199         publicarPoses(0.0f, altura_chasis_, theta_actual_ * (M_PI /
           180.0f), 0, 800);
200
201         // Esperamos medio segundo mecánico para que el gatillo
           realmente se suelte

```

```

202         if ((this->now() - tiempo_fase_).seconds() >= 0.5) {
203             RCLCPP_INFO(this->get_logger(), "Recuperando posici n
                inicial...");
204             estado_actual_ = RECUPERAR_REPOS0;
205             transicion_activa_ = false; // Reseteamos la bandera para
                el siguiente estado
206         }
207     }
208 }
209
210 void Robot::estado_recuperar_reposo()
211 {
212     // Fase 3: Vuelve poco a poco a la posici n de reposo durante 3
        segundos
213     if (!transicion_activa_) {
214         iniciarTransicion(Y_REPOS0, 0.0f, 3.0);
215     }
216
217     // La funci n bloquea l gicamente el estado durante 3s
218     if (actualizarTransicion(altura_chasis_, theta_actual_)) {
219         RCLCPP_INFO(this->get_logger(), "Posici n de reposo alcanzada.
            Secuencia terminada.");
220         estado_actual_ = REPOS0;
221         cambiado_ = true; // Activa la pausa post-estado por seguridad
            si la deseas
222     }
223
224     // Publicamos las poses interpoladas
225     publicarPoses(0.0f, altura_chasis_, theta_actual_ * (M_PI / 180.0f)
        , 0, SERVO_GATILLO_CERRADO);
226 }
227
228 // =====
229 // CALLBACKS ROS
230 // =====
231
232 void Robot::cmd_vel_callback(const geometry_msgs::msg::Twist::SharedPtr
    msg)
233 {
234     bool parado = (msg->linear.x == 0.0 && msg->angular.z == 0.0);
235
236     if (parado) {
237         // Solo volver a REPOS0 si no estamos en alguna fase del salto

```

```

238         bool en_salto = (estado_actual_ == AGACHADO ||
239                         estado_actual_ == PREPARAR_SALTO ||
240                         estado_actual_ == CARGAR_MUELLE ||
241                         estado_actual_ == LIBERAR_MUELLE);
242         if (!en_salto && estado_actual_ != REPOS0) {
243             estado_actual_ = REPOS0;
244         }
245         longitud_paso_ = 0.0f;
246         compensacion_giro_ = 0.0f;
247     } else {
248         estado_actual_ = ANDAR;
249         longitud_paso_ = static_cast<float>(msg->linear.x) * 0.03f;
250         ;
251         compensacion_giro_ = static_cast<float>(msg->angular.z) * 0.002f;
252     }
253 }
254 void Robot::cmd_estado_callback(const std_msgs::msg::Int32::SharedPtr
255                                msg)
256 {
257     switch (msg->data) {
258         case 0:
259             RCLCPP_INFO(this->get_logger(), "    ANDAR");
260             estado_actual_ = ANDAR;
261             cambiado_ = true;
262             break;
263         case 1:
264             RCLCPP_INFO(this->get_logger(), "    AGACHADO");
265             estado_actual_ = AGACHADO;
266             cambiado_ = true;
267             break;
268         case 2:
269             RCLCPP_INFO(this->get_logger(), "    CARGAR_MUELLE");
270             iniciarSecuenciaSalto(h_objetivo_);
271             break;
272         case 3:
273             RCLCPP_INFO(this->get_logger(), "    REPOS0");
274             estado_actual_ = REPOS0;
275             cambiado_ = true;
276             break;
277         default:

```

```

277         RCLCPP_WARN(this->get_logger(), "cmd_estado desconocido: %d
           ", msg->data);
278     }
279 }
280
281 void Robot::cmd_salto_callback(const std_msgs::msg::Float64::SharedPtr
    msg)
282 {
283     iniciarSecuenciaSalto(msg->data);
284 }
285
286 void Robot::salto_listo_callback(const std_msgs::msg::Int32::SharedPtr
    msg)
287 {
288     if (msg->data == 1) {
289         RCLCPP_INFO(this->get_logger(), " Muelle  cargado! Pasando a
            PREPARAR_SALTO.");
290         estado_actual_ = PREPARAR_SALTO;
291         cambiado_ = true;
292     }
293 }
294
295 // =====
296 // HELPER: Iniciar secuencia de salto
297 // =====
298
299 void Robot::iniciarSecuenciaSalto(double h)
300 {
301     h_objetivo_      = h;
302     theta_actual_    = 0.0f;
303     altura_chasis_   = Y_REPOS0;
304
305     // Reset de todas las banderas para garantizar que un segundo salto
        funcione
306     orden_salto_enviada_ = false;
307     transicion_activa_   = false;
308     timer_liberacion_set_ = false;
309     timer_disparo_set_   = false;
310     fase_liberacion_     = 0;
311
312     estado_actual_      = CARGAR_MUELLE;
313     cambiado_           = true;
314     calcularParametrosSalto(h);

```

```

315 }
316
317 // =====
318 // CINEM TICA INVERSA Y TRAYECTORIAS
319 // =====
320
321 void Robot::calcularPataIK(float x, float y, const Pata &p, int &posS1,
    int &posS2) const
322 {
323     // ngulos auxiliares en el tri ngulo formado por los eslabones
324     float alfa1 = atan2(y, x + D_PATA / 2.0f);
325     float alfa2 = atan2(y, x - D_PATA / 2.0f);
326
327     float a1 = std::sqrt((x + D_PATA/2.0f)*(x + D_PATA/2.0f) + y*y);
328     float a2 = std::sqrt((x - D_PATA/2.0f)*(x - D_PATA/2.0f) + y*y);
329
330     float b1 = (L1*L1 + a1*a1 - L3*L3) / (2.0f * L1 * a1);
331     float b2 = (L2*L2 + a2*a2 - L4*L4) / (2.0f * L2 * a2);
332
333     // Comprobar l mite f sico (argumento de acos fuera de [-1, 1])
334     if (std::abs(b1) > 1.0f || std::abs(b2) > 1.0f) return;
335
336     float theta1_deg = (alfa1 + std::acos(b1)) * (180.0f / M_PI);
337     float theta2_deg = (alfa2 - std::acos(b2)) * (180.0f / M_PI);
338
339     posS1 = gradosAPasos(p, 1, theta1_deg);
340     posS2 = gradosAPasos(p, 0, theta2_deg);
341 }
342
343 int Robot::gradosAPasos(const Pata &p, int lado, float grados) const
344 {
345     float pasos;
346     int minSeguro, maxSeguro;
347
348     if (lado == 0) { // Servo 2 (inferior)
349         float factor = (p.config.pwm_down_s2 - p.config.pwm_horiz_s2) /
            90.0f;
350         pasos = p.config.pwm_horiz_s2 + (grados - 0.0f) * factor;
351         minSeguro = std::min(p.config.pwm_horiz_s2, p.config.
            pwm_down_s2 - 200);
352         maxSeguro = std::max(p.config.pwm_horiz_s2, p.config.
            pwm_down_s2 + 200);
353     } else { // Servo 1 (superior)

```

```

354         float factor = (p.config.pwm_down_s1 - p.config.pwm_horiz_s1) /
355             -90.0f;
356         pasos      = p.config.pwm_horiz_s1 + (grados - 180.0f) * factor;
357         minSeguro = std::min(p.config.pwm_horiz_s1, p.config.
358             pwm_down_s1 - 200);
359         maxSeguro = std::max(p.config.pwm_horiz_s1, p.config.
360             pwm_down_s1 + 200);
361     }
362
363     return std::clamp(static_cast<int>(pasos), minSeguro, maxSeguro);
364 }
365
366 void Robot::calcularTrayectoriaPaso(float t_local, float longitud,
367     float &out_x, float &out_y) const
368 {
369     if (t_local < FRACCION_AIRE) {
370         // Fase de vuelo: arco senoidal de -L/2 a +L/2
371         float progreso = t_local / FRACCION_AIRE;
372         out_x = -longitud / 2.0f + longitud * progreso;
373         out_y = ALTURA_SUELO - ALTURA_LEVANT * std::sin(progreso * M_PI
374             );
375     } else {
376         // Fase de apoyo: deslizamiento rectilíneo de +L/2 a -L/2
377         float progreso = (t_local - FRACCION_AIRE) / (1.0f -
378             FRACCION_AIRE);
379         out_x = longitud / 2.0f - longitud * progreso;
380         out_y = ALTURA_SUELO;
381     }
382 }
383
384 // =====
385 // HELPER: Publicar poses con inclinación opcional
386 // =====
387
388 /// Calcula la IK para las 4 patas y publica el mensaje de poses.
389 /// @param theta_rad ángulo de inclinación del chasis en radianes (0
390     = plano).
391 /// Las patas 0 y 2 (delanteras) bajan según d_pata
392     tan( ).
393 void Robot::publicarPoses(float x_objetivo, float y_base,
394     float theta_rad,
395     int gatillo_s9, int gatillo_s10)
396 {

```

```

390     std_msgs::msg::Int32MultiArray msg;
391     msg.data.resize(10);
392
393     for (int i = 0; i < 4; i++) {
394         // Las patas delanteras ( ndices 0 y 2) se ajustan seg n la
           inclinaci n
395         float y_ik = (i == 0 || i == 2)
396                     ? y_base - D_PATA * std::tan(theta_rad)
397                     : y_base;
398
399         int s1, s2;
400         calcularPataIK(x_objetivo * misPatas_[i].inversor_x, y_ik,
           misPatas_[i], s1, s2);
401         msg.data[i*2] = s1;
402         msg.data[i*2+1] = s2;
403     }
404
405     msg.data[8] = gatillo_s9;
406     msg.data[9] = gatillo_s10;
407     pub_poses_->publish(msg);
408 }
409
410 // =====
411 // GESTI N DE TRANSICIONES TEMPORALES
412 // =====
413
414 void Robot::iniciarTransicion(float y_fin, float theta_fin, double
           duracion_segundos)
415 {
416     y_inicial_ = altura_chasis_;
417     theta_inicial_ = theta_actual_;
418     y_final_ = y_fin;
419     theta_final_ = theta_fin;
420     duracion_transicion_ = duracion_segundos;
421     tiempo_inicio_transicion_ = this->now();
422     transicion_activa_ = true;
423 }
424
425 bool Robot::actualizarTransicion(float &out_y, float &out_theta)
426 {
427     if (!transicion_activa_) return true;
428

```

```

429     double transcurrido = (this->now() - tiempo_inicio_transicion_).
        seconds();
430
431     if (transcurrido >= duracion_transicion_) {
432         // Hemos terminado el tiempo asignado
433         out_y = y_final_;
434         out_theta = theta_final_;
435         transicion_activa_ = false;
436         return true;
437     }
438
439     // Calculamos el progreso entre 0.0 y 1.0
440     float progreso = static_cast<float>(transcurrido /
        duracion_transicion_);
441
442     // Interpolación lineal
443     out_y = y_inicial_ + progreso * (y_final_ - y_inicial_);
444     out_theta = theta_inicial_ + progreso * (theta_final_ -
        theta_inicial_);
445
446     return false; // A n estamos en transición
447 }
448
449 // =====
450 // C L CULO DE PAR METROS DE SALTO
451 // =====
452
453 void Robot::calcularParametrosSalto(double h_objetivo)
454 {
455     // Rango de b squeda del ngulo de lanzamiento
456     constexpr int RESOLUCION = 1000;
457     const double theta_min = std::atan(1.0 / MU_FRICCION);
458     const double theta_max = (M_PI / 2.0) - 0.01;
459     const double paso_theta = (theta_max - theta_min) / (RESOLUCION -
        1);
460
461     double v0_sq_min = std::numeric_limits<double>::infinity();
462     double theta_opt = 0.0;
463
464     // B squeda del ngulo que minimiza la velocidad de despegue
        necesaria.
465     // Con d_min == d_max la b squeda es trivial; se mantiene el
        doble

```

```

466 // bucle por compatibilidad con futuras variaciones de la distancia
467 .
468 for (int i = 0; i < RESOLUCION; ++i) {
469     double theta      = theta_min + i * paso_theta;
470     double cos2        = std::cos(theta) * std::cos(theta);
471     double tan_theta   = std::tan(theta);
472
473     // El Z0 de despegue depende de la inclinaci n del robot con
474     // la pata extendida
475     double z0_takeoff = LONGITUD_PATA_SALTO * std::sin(theta);
476     double denom      = z0_takeoff + DIST_SALTO * tan_theta -
477                         h_objetivo;
478
479     if (denom <= 0.0) continue; // La trayectoria no alcanza la
480     // altura
481
482     double v0_sq = (DIST_SALTO * DIST_SALTO * G) / (2.0 * cos2 *
483     // denom);
484     if (v0_sq < v0_sq_min) {
485         v0_sq_min = v0_sq;
486         theta_opt = theta;
487     }
488 }
489
490 if (std::isinf(v0_sq_min)) {
491     RCLCPP_ERROR(this->get_logger(),
492     // "Ninguna trayectoria alcanza h = %.2f m. Salto
493     // cancelado.", h_objetivo);
494     return;
495 }
496
497 double v0 = std::sqrt(v0_sq_min);
498
499 // Convertir a ngulo de inclinaci n del chasis (complementario
500 // al de lanzamiento)
501 theta_opt_deg_ = 90.0 - theta_opt * (180.0 / M_PI);
502
503 // Compresi n necesaria del muelle
504 double m_g_sin = MASA_ROBOT * G * std::sin(theta_opt);
505 double delta_x = (m_g_sin + std::sqrt(m_g_sin*m_g_sin + 2*
506     EFICIENCIA*(K_MUELLE_1 + K_MUELLE_2) * MASA_ROBOT * v0_sq_min))
507     / (2*EFICIENCIA*(K_MUELLE_1 + K_MUELLE_2));
508
509

```

```

500     y_prep_salto_ = (LONGITUD_PATA_SALTO - delta_x) * std::sin(
501         theta_opt);
502     if (y_prep_salto_ < 0.02) y_prep_salto_ = 0.02;
503
504     // --- Resultados de diagnóstico ---
505     RCLCPP_INFO(this->get_logger(),
506         "SALTO: v0=%.2f m/s | incl=%.1f | dist=%.2f m | x_muelle
507         =%.4f m | Y_prep=%.3f m",
508         v0, theta_opt_deg_, DIST_SALTO, delta_x, y_prep_salto_);
509
510     if ((delta_x*100) >= 2.4)
511     {
512         RCLCPP_WARN(this->get_logger(), "Compresión requerida (%.1f cm
513         ) excede el muelle. Salto NO viable.", delta_x * 100.0);
514         movimiento_servo_ = 0.0;
515     } else {
516         // Tiempo de activación del servo
517         double t_servo_s = (Z2*Z4*Z_CARRETE*1023*delta_x) / (MG_0_1023
518             * Z1*Z3*Z5 * W_MAX_SERVO * R_CARRETE);
519         movimiento_servo_ = t_servo_s*1000;
520     }
521
522     // =====
523     // MAIN
524     // =====
525
526     int main(int argc, char **argv)
527     {
528         rclcpp::init(argc, argv);
529         rclcpp::spin(std::make_shared<Robot>());
530         rclcpp::shutdown();
531         return 0;
532     }

```




Código del nodo esclavo ESP32

Listing C.1: Fragmento principal del nodo micro-ROS en ESP32

```

1 // Callback de la petición de tensado (servo 9, modo motor)
2 void subscription_salto_callback(const void * msgin)
3 {
4     const std_msgs__msg__Int32 * msg =
5         (const std_msgs__msg__Int32 *)msgin;
6     long tiempo_pedido = msg->data;
7
8     if (!ejecutando_salto && tiempo_pedido != 0) {
9         ejecutando_salto = true;
10        duracion_salto_ms = abs(tiempo_pedido);
11        tiempo_inicio_salto = millis();
12        sc.WritePWM(9, (tiempo_pedido > 0) ? 700 : 2000);
13    }
14 }
15
16 void loop() {
17     if (ejecutando_salto) {
18         if (millis() - tiempo_inicio_salto >= duracion_salto_ms) {
19             sc.WritePWM(9, 0); // Freno
20             ejecutando_salto = false;
21             msg_listo.data = 1;
22             rcl_publish(&publisher_listo, &msg_listo, NULL);
23         }

```

```
24     }  
25     rclcpp_executor_spin_some(&executor, RCL_MS_TO_NS(10));  
26 }
```



Comandos de referencia rápida

Listing D.1: Comandos útiles para lanzar el sistema

```
1  # Lanzar el agente micro-ROS (conectar ESP32 por serie)
2  ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyUSB0
3
4  # Lanzar el nodo principal
5  ros2 run tfg_salto tfg_robot
6
7  # Ordenar un salto sobre un obstaculo de 45 cm de altura
8  ros2 topic pub -1 /cmd_salto std_msgs/msg/Float64 "{data: 0.05}"
9
10 # Lanzar la teleoperacion por teclado
11 python3 teleop_teclado.py
```


Referencias

- [1] R. Armour, K. Paskins, A. Bowyer, J. Vincent, and W. Megill, “Jumping robots: a biomimetic solution to extreme terrain,” *Bioinspiration & biomimetics*, vol. 2, no. 3, p. S65, 2007.
- [2] P. Fiorini and J. W. Burdick, “The development of hopping mobility for miniature interplanetary rovers,” in *Proceedings of the 2003 IEEE International Conference on Robotics and Automation (ICRA)*, vol. 1. IEEE, 2003, pp. 155–160.
- [3] M. H. Raibert, *Legged robots that balance*. MIT press, 1986.
- [4] D. W. Haldane, M. M. Plecnik, J. K. Yim, and R. S. Fearing, “Robotic vertical jumping agility via series-elastic power modulation,” *Science Robotics*, vol. 1, no. 1, p. eaag2048, 2016.
- [5] E. W. Hawkes, C. Murphy, H. Zhu *et al.*, “Engineered jumpers overcome biological limits via work multiplication,” *Nature*, vol. 604, no. 7907, pp. 657–661, 2022.
- [6] T. Chen, O. R. Bilal, K. Shea, and C. Daraio, “Harnessing bistability for rapid actuation of soft robots,” *Proceedings of the National Academy of Sciences*, vol. 115, no. 22, pp. 5698–5702, 2018.
- [7] M. T. Tolley, R. F. Shepherd, M. Karpelson, N. W. Bartlett, P. Dickson, R. J. Wood, and G. M. Whitesides, “An untethered jumping soft robot,” in *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2014, pp. 400–405.
- [8] Q. Nguyen, M. J. Powell, B. Katz, J. Di Carlo, and S. Kim, “Optimized jumping on the mit cheetah 3 robot,” in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 7448–7454.
- [9] M. Hutter, C. Gehring, D. Jud, A. Lauber, C. D. Bellicoso, V. Tsounis, J. Hwangbo *et al.*, “Anymal-a highly mobile and dynamic quadrupedal robot,” in *2016 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2016, pp. 38–44.
- [10] T. J. Roberts and E. Azizi, “Flexible mechanisms: the diverse roles of biological springs in vertebrate movement,” *Journal of Experimental Biology*, vol. 214, no. 3, pp. 353–361, 2011.

- [11] M. Ilton, M. S. Bhamla, X. Ma, S. M. Cox, L. L. Fitchett, Y. Kim, J.-s. Koh, D. Krishnamurthy, C.-Y. Kuo, F. Z. Temel *et al.*, “The principles of cascading power limits in small, fast biological and engineered systems,” *Science*, vol. 360, no. 6387, p. eaao1082, 2018.
- [12] S. J. Longo, S. M. Cox, E. Azizi, M. Ilton, P. S. Anderson, and S. Patek, “Beyond power amplification: latch-mediated spring actuation is an emerging framework for the study of diverse elastic systems,” *Journal of Experimental Biology*, vol. 222, no. 15, p. jeb197889, 2019.
- [13] J. Zhang *et al.*, “Design of the jump mechanism for a biomimetic robotic frog,” *ResearchGate (Preprint)*, 2022, disponible en la base de datos de biomimética aplicada.
- [14] Novel Mobile Robots Lab, “Pinto: A latched spring actuated robot for jumping and perching,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2026.
- [15] G. Kenneally, A. De, and D. E. Koditschek, “Design principles for a family of direct-drive legged robots,” *IEEE Robotics and Automation Letters*, vol. 1, no. 2, pp. 900–907, 2016.
- [16] N. Kau, A. Schultz, N. Ferrante, and P. Slade, “Stanford doggo: An open-source, quasi-direct-drive quadruped,” in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 6309–6315.
- [17] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.
- [18] eProsima and the micro-ROS Project. micro-ros: Bringing ros 2 to microcontrollers, 2026. Consultado el 14 de junio de 2026. [En línea]. Disponible: <https://micro.ros.org/>
- [19] R. y. o. Fawcett, *Robotic Locomotion: Principios analíticos y simulación cinemática*. Springer Engineering, 2022.
- [20] W. Bolton, *Mechatronics: Electronic control systems in mechanical and electrical engineering*, 6th ed. Pearson Education, 2015.
- [21] Shenzhen Feetech RC Model Co., Ltd., *SCS0009 9g Serial Bus Servo Motor Datasheet*, 2023, especificaciones técnicas y protocolo de comunicación proporcionados por el fabricante.